DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

Ar Limpo, Cidades Saudáveis: Monitorização e Identificação de Poluição com Tecnologia Inteligente

ISEP

2024 / 2025

1121200 Rui Beloto

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Ar Limpo, Cidades Saudáveis: Monitorização e Identificação de Poluição com Tecnologia Inteligente

ISEP

2024 / 2025

1121200 Rui Beloto

Licenciatura em Engenharia Informática

Setembro 2025

Orientador ISEP: **Luís Conceição**

"Ever tried. Ever failed. No matter. Try again. Fail again. Fail better."

- Samuel Beckett

Agradecimentos

Os meus sinceros agradecimentos à estrutura do GECAD por todo o equipamento disponibilizado e em especial ao Prof. Luís Conceição pelo acompanhamento e ajuda. E um agradecimento à Carolina por estar sempre presente na minha vida e por toda a paciência ao longo deste percurso. Um agradecimento à iniciativa AI & Sustainability in VET Education (AISVE), financiado pelo programa Erasmus+ (KA220-VET), com referência 2023-1-LT01-KA220-VET-000155506, que permitiu a viabilização financeira do projecto.

Abstract

Air quality monitoring is a growing challenge due to the increasing presence of pollution sources in urban and rural areas. This project aimed to design and implement a multimodal system capable of combining environmental sensor data with real-time image analysis to provide reliable assessments of air quality.

The solution integrates a Raspberry Pi connected to environmental sensors (CO_2, CO, PM2.5, among others) and a camera, responsible for synchronous data and image acquisition. A dataset was built with real multimodal examples, and a deep learning model was trained and optimized, using NetRes for image analysis and dense layers for the integration of sensor variables. The backend, developed with FastAPI and deployed in the cloud, manages inference, data storage and communication with the client device. A web-based frontend was developed to present results and allow user interaction. The system was evaluated under different scenarios, including nighttime conditions, smoke columns, known pollution sources, and ambiguous cases such as cloudy skies.

The results demonstrate that the multimodal approach increases the robustness of predictions when compared to models based solely on images or sensors. The system achieved 96.59% accuracy on the test dataset, with stable performance in real-time experiments. The developed solution thus meets the proposed objectives, proving to be reliable and adaptable for continuous air quality monitoring.

Keywords (Themes): **Air quality monitoring, multimodal learning, environmental sensors**

Keywords (Technologies): **Raspberry Pi, Arduino, Deep Learning, FastAPI, NetRes, Computer Vision**

Índice

1 Introdução 1

1.1 Enquadramento/Contexto 1

1.2 Descrição do Problema 3

1.3 Estrutura do relatório 5

2 Estado da arte 6

2.1 Trabalhos relacionados 6

2.2 Tecnologias existentes 10

3 Análise e desenho da solução 15

3.1 Análise da solução 15

3.2 Requisitos funcionais, não funcionais e possíveis aplicaçoes 16

3.3 Desenho da solução 18

4 Implementação da Solução 31

4.1 Descrição da implementação 32

4.2 Avaliação da solução 47

5 Conclusões 55

5.1 Objetivos concretizados 55

5.2 Limitações e trabalho futuro 56

5.3 Apreciação final 56

6 Referências 57

Índice de Figuras

Figura 1: Instalação arduino e sensores. 20

Figura 2: Arquitetura do arduino uno [22]. 21

Figura 3: Raspberry pi 4 com câmara v2.0. 22

Figura 4: Arquitetura do raspberry pi 4 [23]. 23

Figura 5: Diagrama geral da solução. 24

Figura 6: Diagrama de componentes. 25

Figura 7: Diagrama de sequência, caso de uso captura e inferência. 25

Figura 8: Diagrama físico do sistema. 26

Figura 9: Diagrama do índice de qualidade do ar [22]. 27

Figura 10: Exemplo imagem capturada na zona da Maia, Portugal. 28

Figure 11: Cidade de Beijing, China, capturado por webcam (WebcamLive 2025). 28

Figura 12: Raspberry pi 4 com câmera v2.0. 32

Figura 13: Instalação dos sensores e arduino uno. 32

Figura 14: Excerto de código com configuração dos sensores no arduino IDE. 34

Figura 15: Output da porta serial do arduino com valores obtidos dos sensores. 35

Figura 16: Exemplo foto capturada para análise. 35

Figure 17: Foto noturna capturada em Bombai, India (WebcamLive 2025). 36

Figura 18: Foto com smog em Beijing, China (WebcamLive 2025). 36

Figure 19: Foto na California, Estados Unidos (WebcamLive 2025). 37

Figure 20: Foto parque da cidade, Matosinhos, Portugal. 37

Figura 21: Excerto do ficheiro csv utilizado com dados dos sensores e associação à imagem respectiva. 38

Figura 22: Excerto código para criar e treinar o modelo de visão computacional. 39

Figura 23: Output dos ciclos de treino do modelo com respectivas perdas associadas. 40

Figura 24: Métricas de avaliação do modelo criado e matriz de confusão. 40

Figure 25: Exemplo código para envio do relatório gerado. 41

Figura 26: Excerto de código para captura da imagem e avaliação por parte do modelo. 42

Figura 27: Excerto de código sobre configuração do modo de inferência. 42

Figura 28: Excerto de código com tempo de verificação das configurações e url para conexão com o servidor.. 42

Figura 29: Envio dos dados dos sensores para o servidor. 43

Figura 30: Interface web desenvolvida sem dados obtidos. 44

Figura 31: Implementação dos botões para captura. 45

Figure 32: Exemplo testes unitários realizados ao backend. 46

Figure 33: Exemplo teste e2e para testar captura do sistema. 46

Figura 34: Relatório de execução de teste com percentagem de cobertura e aprovados/rejeitados. 47

Figura 35: Exemplo de teste do sistema com imagem de ar limpo. 48

Figure 36: Foto com nuvens e céu cinzento. 49

Figure 37: Foto com nuvens e céu cinzento mas, avaliação em modo imagem apenas..... 50

Figura 38: Foto noturna, modo imagem apenas. 50

Figura 39: Foto noturna com modo imagem e dados. 51

Figura 40: Foto com coluna de fumo. 52

Índice de Tabelas

Tabela 1: Planeamento do trabalho 5

Tabela 2: Comparação entre diferentes estudos e solução proposta..... 12

Notação e Glossário

Accuracy (Acurácia) – Métrica que mede a proporção de previsões corretas sobre o total de exemplos avaliados.

Altura (Altitude) – Valor calculado a partir da pressão atmosférica medida pelo sensor BME680.

API (Application Programming Interface) – Conjunto de rotinas e especificações que permite a comunicação entre aplicações; no projeto, o backend expõe uma API REST para o frontend e para o Raspberry Pi.

Arduino – Microcontrolador utilizado para ligação e leitura de sensores analógicos e digitais.

Backend – Parte da aplicação que corre no servidor e trata da lógica de negócio, armazenamento e processamento de dados.

Batch size – Número de exemplos processados em cada iteração de treino antes da atualização dos parâmetros do modelo.

BME680 – Sensor ambiental que mede temperatura, pressão, humidade e resistência de gás.

Cloud – Plataforma remota (neste caso, Render) onde o backend foi colocado em produção.

CO (Monóxido de carbono) – Gás poluente medido pelo sensor MQ-7.

CO_2 (Dióxido de carbono) – Gás poluente medido pelo sensor MG-811.

Confusion matrix (Matriz de confusão) – Tabela que resume o desempenho de um classificador, mostrando verdadeiros positivos, falsos positivos, verdadeiros negativos e falsos negativos por classe.

CPU (Central Processing Unit) – Unidade de processamento central; executa código do sistema e inferências em ambientes sem GPU.

CSV (Comma-Separated Values) – Formato de ficheiro tabular usado para armazenar leituras de sensores e metadata das imagens.

CrossEntropyLoss (Entropia cruzada) – Função de perda comum em problemas de classificação com múltiplas classes, usada durante o treino do modelo.

Data augmentation (Aumento de dados) – Técnicas que geram variantes artificiais de imagens (ex.: rotação, corte) para melhorar a robustez do treino.

Dataset multimodal – Conjunto de dados que combina diferentes modalidades de entrada (imagens e sensores).

Deployment – Processo de colocar a aplicação em produção (no projeto, o backend foi deployado na plataforma Render).

Edge computing – Execução de processamento em dispositivos locais (Raspberry Pi) em vez de servidores remotos.

Epoch (Época) – Uma passagem completa por todo o dataset de treino durante o processo de optimização.

FastAPI – Framework Python utilizada para construir o servidor/backend.

Frontend – Parte da aplicação visível ao utilizador, implementada neste projeto em HTML/JavaScript.

Fusão multimodal (Sensor fusion / Multimodal fusion) – Combinação de informação de diferentes fontes (imagens + sensores).

GPU (Graphics Processing Unit) – Processador especializado usado para acelerar treino e inferência de redes neurais.

Inferência – Processo de usar um modelo treinado para produzir previsões sobre novos dados.

JSON (JavaScript Object Notation) – Formato leve de dados em texto usado para comunicação entre frontend e backend.

LabelEncoder – Ferramenta do scikit-learn para converter labels categóricos em inteiros usados no treino.

Latency (Latência) – Tempo de resposta entre o envio de dados e a receção da predição.

Libcamera / Picamera2 – Biblioteca utilizada no Raspberry Pi para capturar imagens da câmara.

Loss (Perda) – Valor que indica o erro do modelo durante o treino; o objetivo é minimizá-lo.

Média temporal (Temporal averaging) – Cálculo da média das leituras dos sensores ao longo de um intervalo para reduzir ruído.

MG-811 – Sensor analógico usado para medir dióxido de carbono (CO_2).

MobileNetV2 – Arquitetura de rede neuronal convolucional leve e eficiente, usada para extrair características das imagens.

Modelo multimodal – Modelo de aprendizagem que combina dados de diferentes naturezas (imagens e sensores).

MQ-7 – Sensor de monóxido de carbono (CO).

Normalização / StandardScaler – Processo de transformar variáveis para média zero e desvio padrão 1.

Overfitting (Sobreajuste) – Quando o modelo aprende demasiado bem os dados de treino e perde capacidade de generalização.

PM2.5 – Partículas inaláveis com diâmetro ≤ 2.5 µm, relevantes para medir poluição atmosférica.

Polling – Estratégia em que o Raspberry Pi consulta periodicamente o backend para saber se deve capturar dados.

PPM (parts per million) – Unidade de concentração volumétrica usada para gases.

Precision / Recall / F1 – Métricas derivadas da matriz de confusão: precisão, sensibilidade e medida F1.

Preprocessing (Pré-processamento) – Transformações aplicadas a imagens e dados antes de alimentar o modelo.

PyTorch – Biblioteca de deep learning usada no desenvolvimento e treino do modelo.

Req/Resp (Request/Response) – Pedido e resposta HTTP entre cliente (frontend ou Raspberry) e servidor.

REST (Representational State Transfer) – Estilo arquitetural usado na API backend (com métodos HTTP GET/POST).

ResNet – Arquitetura de CNN profunda baseada em blocos residuais, usada como comparação.

RGB – Formato de imagem (Red-Green-Blue) usado para representar imagens a cores.

Sensores ambientais – Conjunto de dispositivos físicos (MG-811, MQ-7, BME680, SEN0233) usados para medir variáveis de qualidade do ar.

Serial (Comunicação Serial) – Protocolo usado para comunicar entre Arduino e Raspberry Pi.

Softmax – Função matemática que converte logits em probabilidades.

StandardScaler – Transformador do scikit-learn usado para normalizar dados de sensores.

Transfer learning (Aprendizagem por transferência) – Técnica que reutiliza redes pré-treinadas e as ajusta ao problema específico.

Training / Validação / Test – Divisão de dados para treino, validação e teste de modelos.

Uvicorn – Servidor ASGI usado para correr a aplicação FastAPI.

VOC (Compostos Orgânicos Voláteis) – Substâncias químicas voláteis que podem contribuir para poluição do ar.

Worker / Thread – Processo ou sub-processo usado para executar tarefas em segundo plano.

Zero-point calibration – Ajuste inicial necessário em alguns sensores de gás (ex.: MG-811) para definir valores de referência.

1 Introdução

A poluição atmosférica constitui um dos maiores desafios ambientais e de saúde pública da atualidade. O aumento da industrialização, do tráfego urbano e da queima de combustíveis fósseis contribui de forma significativa para a libertação de gases tóxicos e partículas em suspensão. A monitorização rigorosa destes poluentes torna-se essencial para prevenir riscos à saúde humana e reduzir impactos ambientais [1].

A investigação recente tem explorado o uso de sensores de baixo custo, técnicas de visão computacional e métodos de inteligência artificial como soluções promissoras para a deteção e caracterização da poluição. Este trabalho insere-se neste contexto, propondo uma abordagem híbrida que combina dados captados por sensores ambientais com imagens associadas a cada recolha. O objetivo é melhorar a precisão da deteção de fontes de poluição visível, como fumo ou emissões industriais, recorrendo a modelos de aprendizagem profunda [2].

Adotou-se uma abordagem baseada em visão computacional, complementada com variáveis contextuais recolhidas por sensores, permitindo a criação de um modelo multimodal. Os principais contributos incluem: Integração de sensores físicos e dados visuais num único pipeline; Desenvolvimento de um modelo de aprendizagem automática capaz de correlacionar leituras ambientais e imagens; Proposta de um sistema que combina simplicidade de implementação e potencial de expansão.

1.1 Enquadramento/Contexto

O projeto insere-se na área da monitorização ambiental e do uso de tecnologias digitais emergentes para avaliação da qualidade do ar. A viabilização financeira do projecto foi realizada através do AI & Sustainability in VET Education (AISVE), financiado pelo programa Erasmus+ (KA220-VET), teve início em setembro de 2023 e decorrerá ao longo de 35 meses, com um orçamento total de 400.000 euros. Reunindo oito parceiros provenientes de sete países europeus – incluindo Portugal, Lituânia, Grécia, Espanha, Turquia, Macedónia do Norte e Bélgica – o consórcio integra universidades, escolas profissionais, associações e empresas, refletindo uma abordagem multidisciplinar e colaborativa.

O objetivo central do AISVE é o desenvolvimento de um sistema abrangente de capacitação em Inteligência Artificial, orientado para os contextos da educação e formação profissional (VET), com especial enfoque na integração de competências ligadas à sustentabilidade e à transição verde. Pretende-se, com esta iniciativa, alinhar as competências de estudantes e

professores com as exigências atuais e futuras do mercado de trabalho europeu, promovendo simultaneamente a inovação pedagógica e o compromisso ambiental.

Espera-se que o projeto contribua de forma significativa para o reforço das competências digitais e verdes nos sistemas de educação e formação profissional, consolidando uma comunidade internacional de prática que promova a articulação entre escolas, universidades, empresas e organizações da sociedade civil. A disponibilização aberta dos recursos desenvolvidos e a sustentabilidade da rede criada garantirão, a longo prazo, a continuidade do impacto, promovendo novas oportunidades de empregabilidade no contexto da transição digital e ecológica na Europa.

A escolha desta temática resulta da relevância social e ambiental do problema e da oportunidade de contribuir para a criação de ferramentas que possam ser aplicadas em diferentes contextos urbanos ou industriais. Ao nível pessoal, os tópicos abordados contribuem para uma possibilidade de expansão de conhecimento sobre tecnologias de visão computacional e possibilidade de desenvolver um trabalho com sistemas integrados e circuitos eletrônicos. Esses factos aliados com a capacidade de expansão do projeto numa solução inovadora tornam o projecto muito apelativo.

1.2 Descrição do Problema

A monitorização da poluição encontra limitações quando baseada apenas em sensores químicos, dado que estes não permitem identificar visualmente a origem das emissões. Por outro lado, a visão computacional aplicada de forma isolada carece de dados complementares que permitam contextualizar as imagens e evitar interpretações ambíguas. O problema em estudo centra-se, assim, na ausência de soluções integradas que combinem leituras sensoriais com informação visual, assegurando uma análise mais robusta da qualidade do ar.

1.2.1 Objetivos

O trabalho enquadra-se no âmbito de monitorização, prevenção e deteção de poluentes aéreos. O objectivo principal pode ser interpretado de diversas formas. Uma vez que, o grande foco é, utilizar visão computacional, criou-se os seguintes tópicos como guia e referência para atingir o objectivo principal.

- Desenvolver um sistema capaz de capturar leituras ambientais (CO_2, CO, PM2.5, temperatura, humidade, VOCs) e associá-las a imagens do local.
- Criar um conjunto de dados multimodal com exemplos reais para treino e validação de modelos de aprendizagem automática.
- Treinar e otimizar um modelo de visão computacional multimodal para identificação de fontes de poluição.
- Avaliar o desempenho do modelo em diferentes cenários, comparando-o com abordagens tradicionais baseadas apenas em sensores ou apenas em imagens.
- Integrar os dados numa interface.
- Elaborar um tutorial para configuração do sistema.

1.2.2 Abordagem

A abordagem assenta na recolha de dados através de um sistema híbrido composto por sensores ambientais (ligados a Raspberry Pi e Arduino) e imagens capturadas de forma síncrona. Foi adotada uma metodologia de aprendizagem multimodal, em que os dados de sensores são integrados com as representações extraídas das imagens através de redes neurais convolucionais [3] [4].

O modelo de base segue práticas descritas na literatura, utilizando arquiteturas pré-treinadas para a componente visual e camadas densas para integração das variáveis sensoriais [3] [4]. Esta abordagem combina a robustez da visão computacional com a precisão dos sensores, reduzindo erros de classificação e aumentando a interpretabilidade dos resultados.

Para além da vertente de modelação, a resolução do problema seguiu uma metodologia iterativa e incremental. Numa primeira fase foi estabelecida a comunicação entre os diferentes componentes (frontend, servidor e Raspberry Pi), garantindo a fiabilidade na recolha e transmissão de dados. Seguidamente, procedeu-se à integração da inferência do modelo no servidor e à implementação de mecanismos de controlo remoto da captura. Por fim, validou-se o sistema em ambiente real, assegurando que os requisitos funcionais estavam cumpridos e que a solução era utilizável de forma contínua.

1.2.3 Contributos

Para este projecto o foco foi desenvolver um mecanismo de baixo custo para monitorização da poluição do ar que envolve visão computacional. Os principais contributos deste trabalho incluem:

- Proposta de uma solução multimodal inédita no contexto de monitorização de poluição visível.
- Desenvolvimento de um dataset original que integra imagens e dados sensoriais de outros datasets com dados originais.
- Criação de um modelo leve e otimizado para execução em dispositivos de baixo custo.
- Demonstração do valor acrescentado da integração de sensores e visão computacional para deteção de poluição.
- Potencial contributo social e ambiental, fornecendo uma ferramenta acessível para monitorização e sensibilização.

1.2.4 Planeamento do trabalho

O planeamento do trabalho foi organizado como apresentado na tabela 1. Surgiram alguns contratempos na aquisição de equipamento que foram contornados ao ajustar as datas do planeamento.

Tabela 1: Planeamento do trabalho

Actividade	**Data início**	**Data fim**
Pesquisa e revisão bibliográfica	23/02/2025	09/03/2025
Aquisição de equipamento	09/03/2025	28/07/2025
Calibração de sensores e configuração das câmaras para captura de imagens	25/08/2025	28/07/2025
Desenvolvimento do modelo de visão computacional	25/08/2025	30/08/2025
Integração de sensores e IA	30/08/2025	31/08/2025
Desenvolvimento da interface	31/08/2025	02/09/2025
Testes com sensores e câmaras em diferentes ambientes	31/08/2025	03/09/2025
Avaliação e otimização do modelo de IA	31/08/2025	03/09/2025
Documentação	25/08/2025	06/09/2025

1.3 Estrutura do relatório

Para além da introdução, esta dissertação contém mais 5 capítulos. No capítulo 2, é descrito o estado da arte e são apresentados trabalhos relacionados. No capítulo 3, é realizado o desenho e modelação da solução proposta, com metodologia, técnicas a usar e equipamentos. No capítulo 4, é avaliado o modelo criado e os resultados obtidos da análise em tempo real da solução obtida. No capítulo 5, é realizada uma conclusão que engloba e resume todas as conclusões obtidas do trabalho final.

2 Estado da arte

A monitorização visual de poluição, particularmente fumo, plumas de chaminés e emissões visíveis, tem um leque de aplicações. Nomeadamente, em conformidade regulatória, detecção precoce de incêndios, controlo industrial e vigilância ambiental urbana. Os métodos tradicionais (observação humana, protocolos como EPA 9/22 ou o artigo 192º e 218º da UE) provêm da necessidade histórica de quantificar a opacidade e frequência de emissões. O avanço em visão computacional e a disponibilidade de sistemas de baixo custo (câmaras, Raspberry Pi, sensores de qualidade do ar de baixo custo) permitem automatizar a observação contínua e a fusão de sinais visuais com leituras de sensores, com ganhos em sensibilidade, custo e escalabilidade.

2.1 Trabalhos relacionados

As preocupações crescentes com as alterações climáticas, a poluição industrial e a saúde pública têm exigido o desenvolvimento de sistemas de monitorização ambiental robustos, escaláveis e em tempo real. Os métodos tradicionais, que frequentemente dependem de sensores físicos (ex.: estações de monitorização da qualidade do ar in-situ), enfrentam limitações de cobertura espacial, custo e velocidade de implementação. Neste contexto, a visão computacional (VC) apresenta-se como uma ferramenta potente que permite a análise, de uma área ampla, ao nível quantitativo, de fenómenos ambientais. Ao obter e processar dados de uma variedade de plataformas de imagem, desde circuitos fechados de televisão (CCTV), veículos aéreos não tripulados (VANTs/drones) e até satélites, as técnicas de VC estão a revolucionar a forma como detetamos, quantificamos e rastreamos as emissões, a deteção de fogos e poluentes [5].

Uma aplicação bem estabelecida da VC na área da conformidade ambiental é a monitorização de fontes pontuais industriais, como chaminés. A prática comum envolve a utilização de câmaras fixas para capturar vídeo ou imagens de intervalómetro (time-lapse) das plumas. Algoritmos avançados segmentam depois a pluma do fundo, analisando as suas propriedades visuais para quantificar a opacidade, um indicador-chave das emissões de partículas, e para detetar eventos anómalos como fugas ou nuvens de poeira. Isto fornece aos reguladores e operadores de centrais dados contínuos e objetivos para relatórios de conformidade e deteção precoce de fugas, ultrapassando as observações humanas esporádicas [3] [4].

A gestão de incêndios florestais foi significativamente avançada pela visão computacional, focando-se na deteção precoce de fumo e chamas para minimizar os tempos de resposta. A prática comum aproveita redes de câmaras CCTV e PTZ (Pan-Tilt-Zoom) colocadas estrategicamente, que fornecem uma cobertura contínua de paisagens propensas a fogos. Modelos de VC, frequentemente baseados em redes neuronais convolucionais (CNNs), são treinados para distinguir plumas de fumo de nuvens, nevoeiro ou poeira, e para identificar a assinatura visual única das chamas [5]. Adicionalmente, VANTs/drones equipados com câmaras são utilizados para vigilância aérea, fornecendo dados em tempo real críticos em terrenos complexos. Esta área é suportada por comunidades de investigação ativas e benchmarks que facilitam o desenvolvimento e comparação de algoritmos de deteção. A intersecção entre a IA e as alterações climáticas, destacada por iniciativas como a Climate Change AI, realça a importância destas tecnologias na mitigação de desastres de origem climática [6].

Uma área de investigação inovadora e em crescimento envolve a utilização de VC para inferir a qualidade do ar urbano, particularmente a poluição relacionada com o tráfego (TRAP). A prática comum, sob o paradigma emergente de "câmaras como sensores", utiliza fluxos de vídeo da infraestrutura urbana existente (ex.: câmaras de tráfego) para estimar os níveis de poluentes. Em vez de medir partículas diretamente, os modelos de VC correlacionam características visuais como a neblina, a degradação da visibilidade, os efeitos de dispersão da luz e até a densidade do tráfego com dados de referência de sensores físicos [7-9].

À escala regional e global, a deteção remota por satélite representa a aplicação mais extensiva da VC para a monitorização ambiental. A prática comum envolve a análise de imagens multiespectrais e hiperespectrais de satélites como o Sentinel-2, PlanetScope e MODIS. Os algoritmos de VC são aptos a mapear fenómenos de larga escala: podem monitorizar o transporte de plumas de fumo através de continentes, delinear cicatrizes de queimadas de incêndios florestais, identificar áreas de combustão ativa e estimar volumes de emissõe [8].

Os atuais desenvolvimentos utilizam modelos de *Deep Learning* (DL) para detetar e analisar as imagens obtidas e as bases de dados coletadas através dos mecanismos explicados anteriormente. Os modelos têm como base classificação, deteção de objectos, segmentação, séries temporais, transformadores e modelos fundamentais.

A classificação de imagem completa constitui a abordagem mais direta e rápida, ideal para a triagem (*screening*) inicial de grandes volumes de dados oriundos de câmaras de vigilância ou satélites. As arquiteturas ligeiras e eficientes, como MobileNet e ResNet, mantêm-se

extremamente úteis para implementação nos dispositivos de *edge computing*, onde os recursos computacionais são limitados [9].

Para aplicações que exigem não só a deteção mas, também a localização espacial precisa da pluma dentro de uma imagem, a deteção de objetos é a tarefa preferencial. Esta é a abordagem mais comum em sistemas baseados em CCTV e UAVs (drones), onde é crucial saber onde está o fumo. Modelos da família YOLO (You Only Look Once), nas suas versões v5, v8 e a mais recente v11, dominam o panorama devido ao seu equilíbrio excecional entre velocidade e precisão. De acordo com os estudos, a adaptação do YOLOv8 para a deteção específica de fumo industrial, demonstra uma performance superior em comparação com outros detetores como SSD (Single Shot MultiBox Detector) e Faster R-CNN em contextos de tempo real [9].

Quando o objetivo é a quantificação precisa ou a análise morfológica da pluma, a segmentação semântica, que gera uma máscara pixel a pixel, é o método mais apropriado. Arquiteturas encoder-decoder como U-Net e DeepLab têm sido o pilar desta tarefa. Contudo, o panorama atual foi revolucionado pela introdução dos Modelos de Base (Foundation Models*) para segmentação. O Segment Anything Model (SAM) da Meta, e a sua variante eficiente FastSAM, emergiram como bases universais de segmentação [10-11]. Vários estudos realizados demonstram o *fine-tuning* e aplicação específica do FastSAM para a segmentação de alta precisão de plumas de fumo, aproveitando a sua robustez e capacidade de generalização [10-11].

A natureza dinâmica do fumo torna a análise temporal crucial para reduzir falsos positivos, particularmente para distinguir plumas de fumo de elementos visuais estáticos ou em movimento lento, como nuvens e nevoeiro. Para tal, arquiteturas capazes de modelar a informação espaço-temporal são essenciais. Abordagens que combinam CNNs 3D ou redes CNN-LSTM (Long Short-Term Memory) são amplamente utilizadas para extrair características espaciais e modelar a sua evolução temporal. Um exemplo paradigmático é a arquitetura SmokeyNet, especificamente concebida e treinada em conjuntos de dados como o FIgLib (Fire Ignition Library), que demonstra uma elevada eficácia na deteção de incêndios incipientes através da análise do movimento do fumo em sequências vídeo [12].

Existem diversos estudos que estudam e aplicam os métodos e técnicas abordados. Um dos estudos propõe pipelines com CNNs e etapas de machine learning para classificar/estimar níveis de poluentes a partir de fotos urbanas [13]. Outro trabalho utiliza vídeos de tráfego para extrair variáveis relacionadas ao fluxo veicular como proxies para poluentes relacionados

ao tráfego (TRAP), produzindo previsões em tempo real [14]. Outro estudo aplica arquiteturas profundas (ex.: ResNet) a imagens móveis (smartphones) para estimar valores de PM e NO_2 [15]. Um estudo realizado em Nanjing, usa câmeras de vigilância que exploram informação espaço-temporal contígua para melhorar robustez das estimativas de monitorização da poluição do ar [16]. Um estudo com tecnologia mais dispendiosa consiste em sistemas baseados em UAVs que visam alta resolução espacial através de sensoriamento móvel e discutem desafios de calibração e fusão com medidas de referência [17].

Ao comparar os estudos para procurar semelhanças nas técnicas e métodos usados verifica-se que os artigos cobrem três categorias de entrada: imagens estáticas, vídeo contínuo e imagens móveis (UAV/smartphone). Imagens estáticas são simples de recolher mas muito sensíveis a variações de iluminação, hora do dia e condições meteorológicas. Vídeo oferece continuidade temporal que ajuda a filtrar ruído instantâneo e a capturar picos relacionados a eventos (ex.: engarrafamentos), ao custo de maior processamento. UAVs e dispositivos móveis aumentam a cobertura espacial, mas introduzem variações de ângulo e de calibração que exigem pipeline de normalização complexa [13-17].

Em termos de modelos e arquiteturas, predominam CNNs para extração de características visuais; alguns trabalhos combinam CNNs com componentes temporais (p.ex. ConvLSTM, LSTM) para gravar dependências temporais. Em termos de maturidade, observa-se a mesma evolução vista noutros domínios: arquiteturas puramente espaciais cedem lugar a modelos híbridos spatio-temporais ou multimodais quando se deseja produção operacional [13-17].

Ao nível de supervisão dos modelos, todos os estudos dependem de medidas de referência (estações fixas, sensores low-cost) para rotular imagens. A sincronização temporal e espacial entre a imagem e a medição é uma fraqueza repetida: delays, diferenças de distância e variações microclimáticas podem introduzir ruído ou viés nos rótulos, prejudicando a generalização [13-17].

Em termos de vantagens e desvantagens, a infra-estrutura de câmeras urbanas ou smartphones pode reduzir os custos em cenários com poucas estações de referência. As imagens e vídeo capturam efeitos visuais (partículas em suspensão) que não aparecem diretamente em satélite de baixa resolução. Com pipelines de vídeo/edge computing podem fornecer estimativas locais em quase tempo real, úteis para alertas imediatos. Algumas das limitações são a relação entre indicadores visuais (visibilidade, tonalidade do céu) e concentrações químicas (PM2.5, NO_2) é imperfeita. A existência de diferentes condições meteorológicas podem produzir sinais visuais semelhantes para concentrações díspares.

Alguns dos modelos treinados numa cidade/estação frequentemente degradam noutras localidades por diferenças arquitetónicas, clima, e composição das fontes [13-17].

A prática dominante atual combina redes de estações de referência, sensores low-cost, dados satelitais e modelos spatio-temporais (por exemplo GCNs, ConvLSTM) para mapear e prever poluição. Os estudos baseados exclusivamente em imagens apresentam provas de conceito valiosas mas, na literatura mais avançada, as imagens tendem a ser usadas como uma das várias entradas num sistema multimodal e não como substituto direto das medições.

2.2 Tecnologias existentes

O objetivo é monitorizar, avaliar e quantificar os índices de poluição atráves de VC. Uma outra vertente dos objectivos propostos é fazê-lo com uma solução de baixo custo económico e tecnológico para ser fácil de implementar. Nesta vertente, existem alguns trabalhos realizados que evidênciam as tecnologias existentes e utilizadas em casos de sistemas de baixo custo.

Bogdanffy et al. (2025) e Zhao et al. (2023) destacam-se pela sua validação em ambiente real. Bogdanffy et al. propuseram uma plataforma para monitorização urbana que combina de forma pragmática a contagem de veículos via object detection (usando bibliotecas standard) com leituras de sensores de PM de baixo custo, implementada em nodes de edge computing (ex: Raspberry Pi) [18]. A sua principal contribuição reside na demonstração de custo-efetividade e na arquitetura de pipeline robusta (aquisição local para agregação para cloud), validada através de comparação com estações oficiais. Da mesma forma, Zhao et al. focaram-se no ambiente interior, projetando e implementando um detector multi-parâmetro estável para poluentes como CO_2, PM e VOC [19]. A sua força reside na minuciosa avaliação da estabilidade a longo prazo (um ano) e no design modular de hardware. Contudo, uma limitação comum a ambos os estudos é o foco na plataforma de aquisição e fusão de dados, em detrimento da exploração de algoritmos de visão computacional de estado da arte (SOTA) para análise aprofundada da imagem (ex.: segmentação de plumas).

O trabalho seminal de Zhang et al. (2020) forneceu a prova de conceito crucial, demonstrando que funcionalidades extraídas por CNNs de imagens de telemóvel podiam ser correlacionadas com medições de PM2.5 [20]. A sua abordagem leve, utilizando transfer learning, abriu caminho para aplicações crowdsourced. No entanto, a dependência de uma imagem única torna-o vulnerável a variáveis de confusão como condições de iluminação e meteorologia.

Avançando significativamente nesta linha, o mais recente trabalho de 2025 propõe o AQP-Mamba, um modelo de deep learning espaço-temporal que representa o estado da arte em

métodos video-based [21]. Ao contrário de Zhang et al., que usavam imagens estáticas, o AQP-Mamba processa sequências de vídeo, capturando a dinâmica temporal do haze e do movimento das partículas. A sua arquitetura inovadora, que integra um Structured Selective State Space Model (SSM), é especificamente desenhada para modelar dependências de longo prazo em séries temporais vídeo, melhorando substancialmente a robustez sobre métodos baseados em imagens únicas. A sua principal força é a demonstração de que a riqueza da informação temporal no vídeo pode superar as limitações de iluminação e oferecer estimativas mais fiáveis de AQ, porventura aproximando-se da precisão de sistemas híbridos [21]. A principal contrapartida é o custo computacional inerente ao processamento de vídeo, levantando desafios para a implantação em edge sem otimizações avançadas.

Tabela 2: Comparação entre diferentes estudos e solução proposta.

Artigo	**Objectivo**	**Dados**	**Métodos principais (CV/ML)**	**Hardware / Deployment**	**Principais contribuições**
Bogdanffy et al.	Tráfego e qualidade do ar	Imagem + Sensores	CV + Agregação + IoT	Raspberry pi	Arquitetura integrada IoT+CV; validação in-field; custo-efetividade
Maqsood Ahmed et al.	Prever qualidade ar dinâmica	Estações + Video	CNN + SSM	GPU	Usa dinâmica temporal para melhorar precisão AQ; modelo específico para vídeo
Zhang et al.	Avaliar imagens de telemóvel	Imagem + Estações	CNN + transfer learning	Smartphone	Demonstra viabilidade de prever PM usando imagens únicas; pipelines leves
Zhao et al.	Monitorização indoor	Sensores	I2C + IoT	Dispositivo IoT	Solução multi-parâmetro low-cost e validada 1 ano
Nossa solução (proposta), NetRes + MLP sensores	Multimodal exterior	Imagem + Sensores	NetRes + I2C + Digital	Raspberry pi e arduino	Multimodal, leve, desenhado para edge, deploy testado

A tabela 2 compara os estudos de equipamentos de baixo custo com a nossa solução proposta. A abordagem multimodal adotada na nossa solução destaca-se pela simplicidade e eficácia comprovada. Através de uma estratégia de late fusion, combinam-se funcionalidades extraídas de imagens por uma rede NetRes com dados de sensores de PM, CO_2 e CO processados por uma MLP. Esta combinação permite aproveitar sinais complementares: em

casos onde a imagem é ambígua (e.g., distinção entre nuvem, vapor ou fumo), as leituras dos sensores tendem a esclarecer a natureza da pluma, aumentando a robustez do sistema.

A arquitetura é leve e executável em dispositivos de edge computing, como o raspberry pi. A utilização de uma NetRes pré-treinada e congelada para extração de características, seguida de um classificador simples, garante eficiência computacional. A conversão do modelo para TorchScript permite uma execução razoavelmente rápida num Raspberry Pi mesmo sem utilização de GPU. Esta eficiência contrasta favoravelmente com soluções baseadas em vídeo e modelos espaço-temporais mais complexos, como o AQP-Mamba mencionado na literatura, que exigem substancialmente mais recursos computacionais.

O pipeline implementado é prático e baseado em hardware acessível. A solução recorre a um Arduino para aquisição de dados de sensores analógicos, uma opção robusta e económica que evita a necessidade de conversores ADC adicionais, e a um Raspberry Pi para aquisição de imagem e inferência. Esta configuração hardware é amplamente validada em trabalhos similares, como os descritos por Bogdanffy et al. e Zhao et al., demonstrando ser suficiente para prototipagem e implementação de sistemas integrados de monitorização.

Em termos de limitações, relaciona-se com o não aproveitamento de informação temporal de vídeo. Trabalhos recentes, como o AQP-Mamba, recorrem a State Space Models (SSMs) ou redes 3D-CNN para modelar clips de vídeo, capturando a dinâmica temporal do movimento das plumas. Esta modelação temporal pode reduzir significativamente os falsos positivos, particularmente em casos onde o movimento é um fator discriminativo crucial (e.g., fumo vs. névoa). Para cenários com câmaras capazes de capturar vídeo e recursos computacionais superiores, a incorporação de temporalidade seria uma melhoria natural [9].

Outra limitação reside na ausência de segmentação pixel-wise. Enquanto a solução se foca na classificação de imagem completa ou regressão de valores, abordagens baseadas em U-Net, DeepLab, ou modelos de base como o Segment Anything Model (SAM) permitem obter máscaras de segmentação precisas da pluma. Estas máscaras são indispensáveis para estimativas quantitativas de parâmetros como opacidade ou área de dispersão, aproximando-se dos protocolos standard como o Método 9 da EPA. Se a métrica de opacidade for um requisito, a segmentação é incontornável [19].

Adicionalmente, a dependência de sensores de baixo custo introduz um desafio de calibração. Tal como destacado nos estudos analisados, a fiabilidade das leituras destes sensores está diretamente ligada a processos rigorosos de calibração e comparação com equipamentos de referência. Sem esta etapa, o modelo de visão computacional pode aprender enviesamentos

sistemáticos presentes nos dados brutos, comprometendo a generalização e precisão das previsões. É, portanto, imperativo implementar rotinas de calibração e normalização dos sinais dos sensores antes do treino.

A opção da solução escolhida privilegia a simplicidade, eficiência e integração multimodal e está diretamente alinhada com o objetivo principal do projeto: criar uma solução prática, operável num Raspberry Pi, e de implementação rápida. O trade-off consciente entre complexidade e desempenho é explicitamente recomendado pela literatura para sistemas de edge computing com restrições de recursos.

3 Análise e desenho da solução

Este capítulo permite demonstrar a idealização da solução criada e explicação detalhada da sua construção, implementação e otimização. Serão ainda descritas todas as metodologias e técnicas utilizadas, assim como, todo o equipamento e software.

3.1 Análise da solução

A avaliação da qualidade do ar representa um pilar fundamental para a saúde pública e para a monitorização ambiental. Tradicionalmente, esta tarefa depende de estações de medição fixas, complexas e economicamente dispendiosas, que não conseguem fornecer uma visão granular e hiperlocal da poluição atmosférica. Esta lacuna operacional limita a nossa capacidade de responder de forma eficaz às flutuações da qualidade do ar, que podem variar significativamente de quarteirão para quarteirão. A visão computacional surgiu como uma tecnologia promissora capaz de ajudar, ao inferir sobre os parâmetros de poluição através da análise de imagens. No entanto, a sua aplicação prática tem uma limitação fundamental. A correlação entre o que a câmara capta, como a presença de névoa ou a cor do céu, e as concentrações reais de poluentes específicos é intrinsecamente complexa e não linear. Um céu acinzentado pode dever-se a humidade elevada e não a partículas poluentes, por exemplo.

Numa tentativa de melhorar os avanços da tecnologia para deteção da poluição, utilizamos um sistema multimodal. A premissa central da nossa abordagem é que a combinação de dois tipos de dados, cria uma sinergia que supera as limitações de cada modalidade isolada. Por um lado, as imagens fornecem um contexto ambiente rico e intuitivo. Por outro, os sensores entregam medições quantitativas, objetivas e diretamente relacionadas com os poluentes em causa. Ao fundi-los num único modelo, permitimos que o sistema aprenda a relacionar padrões visuais subtis com concentrações específicas, criando um classificador mais robusto e inteligente.

Todas as opções técnicas foram ponderadas para equilibrar o custo, a complexidade e a precisão. A espinha dorsal de hardware assenta num ecossistema low-cost, mas capaz, composto por um Raspberry Pi 4 e um Arduino Uno. Esta dupla permite uma divisão de tarefas, o Arduino, uma plataforma robusta e ideal para a aquisição de dados analógicos, foi encarregue de gerir e ler os valores dos sensores de CO, CO_2 e PM2.5. O Raspberry Pi, um computador de plena capacidade, assumiu o papel de cérebro do sistema, coordena a captura

de imagens, recebe os dados do Arduino via USB e envia para o modelo de inteligência artificial. A escolha de sensores da DFRobot foi guiada pela relação custo-eficácia e pela fiabilidade demonstrada em projetos da comunidade.

3.2 Requisitos funcionais, não funcionais e possíveis aplicaçoes

Requisitos Funcionais:

- O sistema deve ser capaz de adquirir dados dos sensores e processar imagens, executando a inferência do modelo multimodal em menos de 5 segundos por amostra, permitindo uma resposta quase imediata a eventos de poluição.
- Deve ser disponibilizada uma interface web acessível via browser, atualizada automaticamente, que apresente a imagem mais recente, as leituras dos sensores, o resultado da inferência e o histórico de eventos.
- O sistema deve ser capaz de gerar e publicar automaticamente alertas (ex: via MQTT, email ou API) quando os níveis de poluição detetados excederem limiares pré-definidos configuráveis pelo utilizador.
- A solução deve funcionar de forma autónoma e completa sem dependência de uma ligação à Internet contínua, armazenar dados locais e sincronizar quando a conectividade for restabelecida.

Requisitos Não Funcionais:

- O sistema completo (Raspberry Pi, sensores e periféricos) deve ser optimizado para um consumo energético mínimo, permitindo a sua operação contínua 24/7, potencialmente alimentado por fontes renováveis (ex: painel solar).
- A solução deve ser construída com componentes de hardware off-the-shelf e de baixo custo, tornando-a acessível para implementação widespread por municípios, comunidades ou pequenas indústrias. O custo alvo por unidade é inferior a 300€.
- O sistema deve ser resiliente a falhas intermitentes de sensores, erros de leitura ou reinícios inesperados, retomando a operação normal automaticamente sem perda crítica de dados ou funcionalidade.
- A arquitetura de software e comunicação (ex: MQTT) deve permitir a integração futura de múltiplos nós de monitorização numa rede, com gestão centralizada de dados.

Possiveis aplicações:

Os governos e autoridades reguladoras podem beneficiar como uma ferramenta de fiscalização ambiental acessível. Permite a monitorização de zonas industriais específicas e a verificação do cumprimento de normas de emissões, fundamentais para a definição de políticas públicas informadas.

Indústria pode utilizar a solução para monitorização interna de emissões, gestão proativa do impacto ambiental e demonstração de conformidade com regulamentos, potencialmente reduzindo multas e melhorando a imagem corporativa.

3.3 Desenho da solução

As especificações, técnicas e métodos utilizados para o desenvolvimento da nossa solução alinham com a visão inicial de criar um sistema de visão computacional capaz de detetar poluição do ar. Nas secções seguintes, expõem-se os diferentes equipamentos, métodos e tecnologias pensadas para cumprir os objectivos propostos.

3.3.1 Lista de equipamentos, componentes e software

Os equipamentos foram pensados para serem opções de baixo custo e personalizáveis. Para obtermos os valores de concentração de diferentes gases um sensor único não teria a mesma precisão que sensores especificos. Pesquisou-se inúmeros sensores e os da DFRobot tinham o nível de precisão e preço que estavamos à procura. A seguinte lista enumera todos os equipamentos, hardware e software a utilizar na nossa solução.

Os equipamentos utilizados na nossa solução são os descritos:

- Raspberry Pi 4 Model B (4GB).
- Arduino Uno, alimenta sensores analógicos e comunica por USB com o Pi.
- Cartão microSD (32GB+), sistema operativo, armazenamento local CSV para logs.
- Fonte de alimentação, 5V para Raspberry Pi e fonte externa 7–12V para Arduino e bateria de 5V para potenciar ligaçao a um sensor.
- Cabos USB A-B (Arduino para Pi), cabos jumper dupont, fios para alimentação.

Ao nível de sensores:

- DFRobot SEN0159 (MG811), sensor de CO_2.
- DFRobot SEN0132 (MQ7), sensor de CO.
- DFRobot SEN0248 (BME680), sensor de VOC/Temp/Hum/Press.
- DFRobot SEN0233, sensor de PM2.5.
- Raspberry Pi Camera v2.0, captura imagens do local (imagem única por evento ou frames quando necessário).

Para software e utilitários:

- Raspbian / Raspberry Pi OS.
- Python 3 com múltiplas librarias como linguagem base do projecto devido à facilidade de integração entre componentes e modelo de computer vision.

- Arduino IDE.
- FastAPI para rapidamente desenvolver uma API de comunicação entre backend e frontend.
- Servidor de treino (PC com GPU) com PyTorch completo, scikit-learn, Optuna, MLflow/W&B, DVC (para dataset versioning).
- Servidor em cloud (Render) para deploy e gestão do frontend/backend.

Toda a escolha de equipamentos foi focada numa solução económica, de fácil acesso e substituição de componentes. Privilegiou-se a simplicidade e fiabilidade para focar no objectivo final em vez de uma solução mais robusta com mais funcionalidades e equipamentos.

3.3.2 Ligações físicas e esquema de construção

As ligações dos equipamentos são descritas através das especificidades de cada conexão e de acordo com as fichas técnicas dos sensores.

A primeira montagem é dos sensores ao arduino uno. Optamos pelo arduino, devido à sua versatibilidade e compatibilidade com os sensores. Outra vantagem do arduino é o seu ambiente de desenvolvimento que permite alterar e configurar os sensores de acordo com aquilo que é desejado.

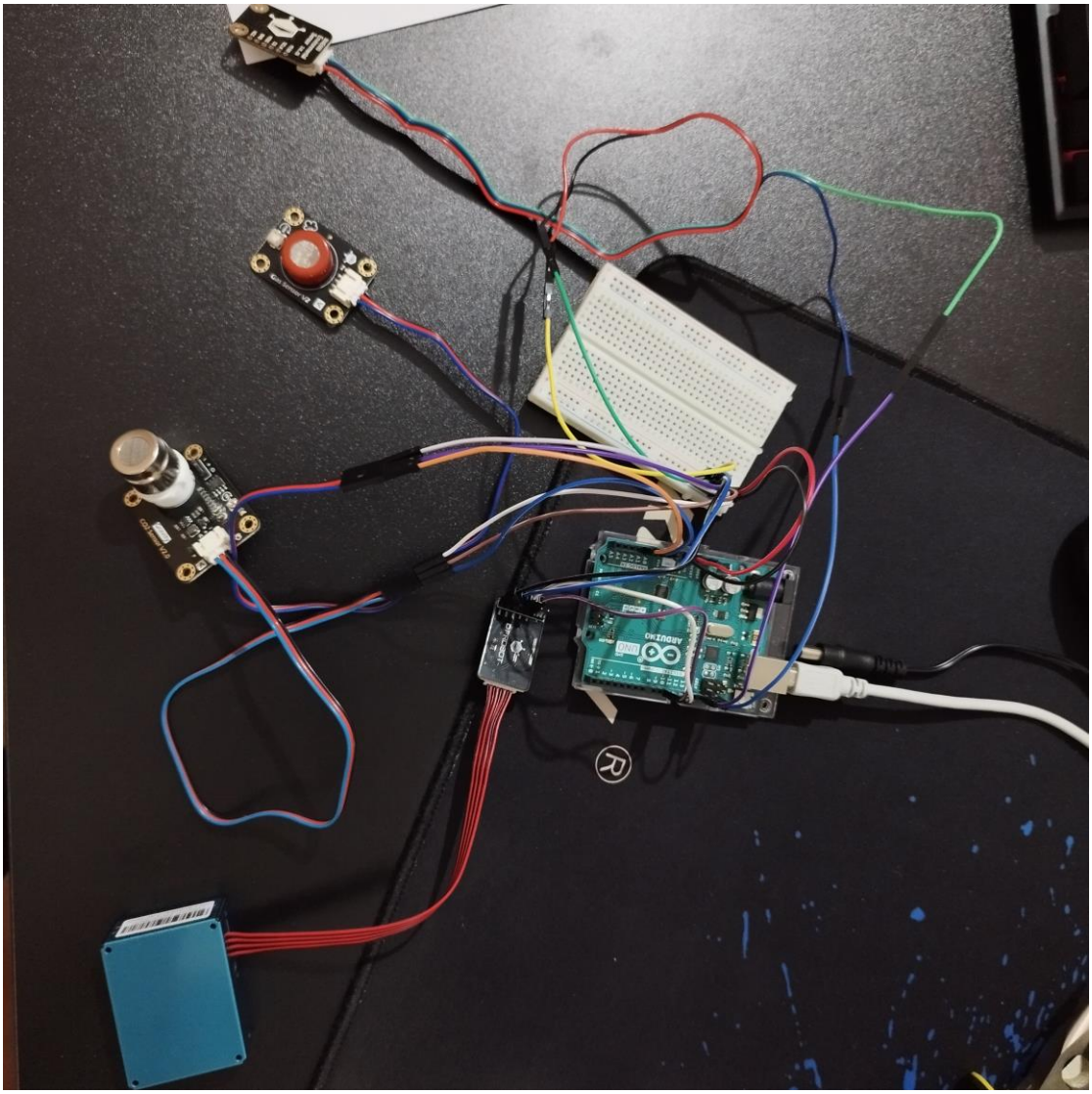

Figura 1: Instalação arduino e sensores.

Os sensores têm um ponto em comum, um fio de alimentação de 5 V e um fio de terra. No arduino, a conexão foi realizada no pino de VCC (5 V) para o fio de fase e no pino de GND para o fio de terra. Uma vez que, temos apenas uma saída de 5 V na placa do arduino uno optou-se por utilizar uma placa de conexão para ligar os 4 sensores em paralelo, nos fios VCC e GND. O circuito interno do arduino não permite uma intensidade de corrente superior a 1 A e uma vez que a intensidade total com os sensores é de 0.6 A não existem complicações ao executar dessa forma. Na figura 1 é possível verificar as ligações descritas.

Na imagem, de baixo para cima, está o sensor de PM2.5, o sensor de CO2, o sensor de CO e o sensor de COVs. Todos estão a convergir na PCB no que toca a VCC e GND para uma ligação em paralelo. No arduino uno liga à saida de 5 V e GND, respectivamente.

Quanto às ligações dos fios de comunicação dos sensores alguns foram conectados por entradas analógicas e outros digitais. A diferença está no próprio sensor, por exemplo, o SEN0132 comunica de forma analógica e usa a diferença obtida na voltagem para extrapolar a presença de CO no ar que o envolve. O sensor SEN0233 já faz a avaliação e quantificação da PM e transmite em sinal digital o valor final.

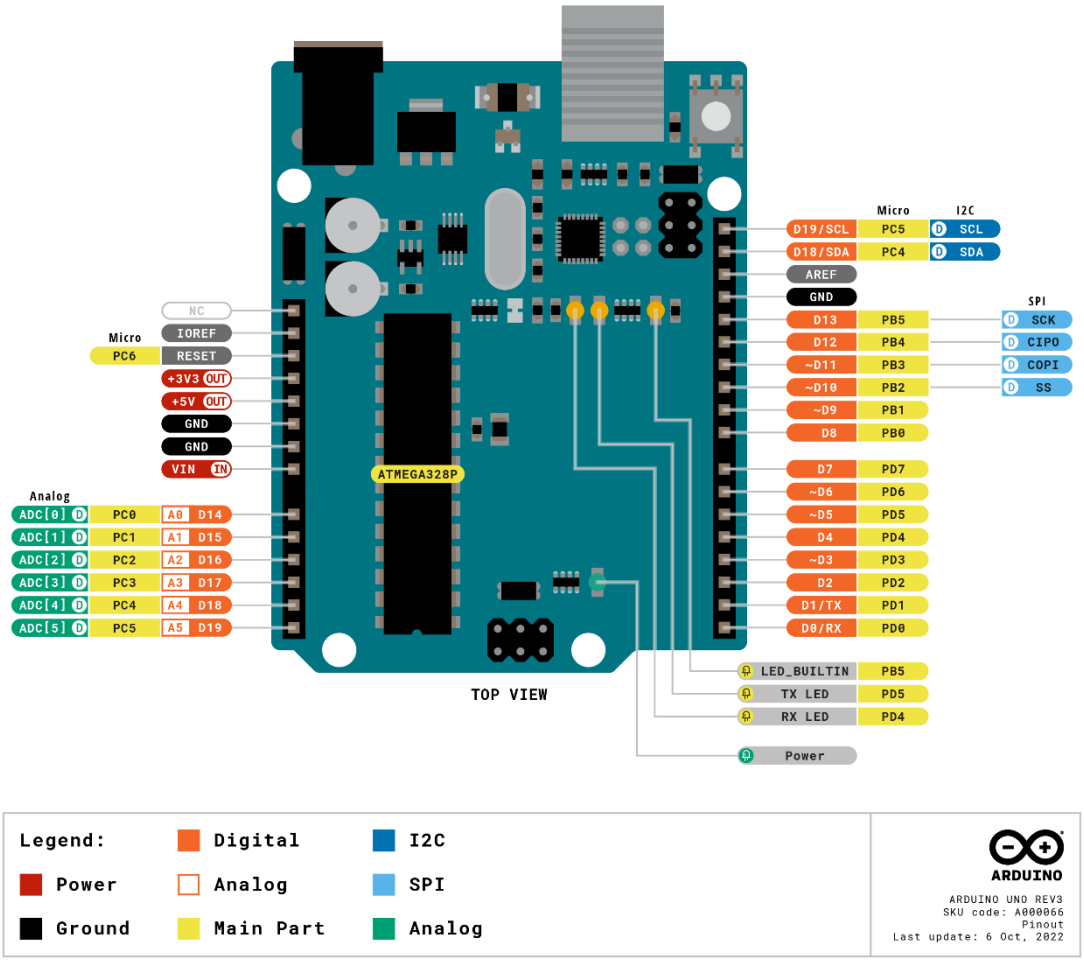

Figura 2: Arquitetura do arduino uno [22].

A figura 2 mostra a arquitetura do arduino uno. As entradas de ADC gerem os sinais analógicos e as D os sinais digitais. Normalmente, um sinal digital tem mais ligaçoes que um analógico, visto que, necessitam de uma entrada de dados e uma entrada de saida de dados.

Para os sensores comunicarem, de CO e CO2, utiliza-se a entrada A0 e A1. Para as entradas digitais, para o sensor de COVs, conectou-se a entrada SCL e SDA e para o sensor de PM2.5 conectou-se a entrada D10 e D11.

O raspberry pi foi planeado como o dispositivo de receção dos dados obtidos pelos sensores e que executa a captura das imagens através da sua camera. O arduino uno conecta ao raspberry pi através de um cabo usb pelo qual os dados são partilhados.

Para alimentar a potência de ambos dispositivos utilizou-se uma fonte de alimentação de 5V para o raspberry pi e para o arduino uma fonte de 12V devido à necessidade de aquecimento dos sensores SEN0159 e SEN0132 (de acordo com a ficha técnica dos sensores).

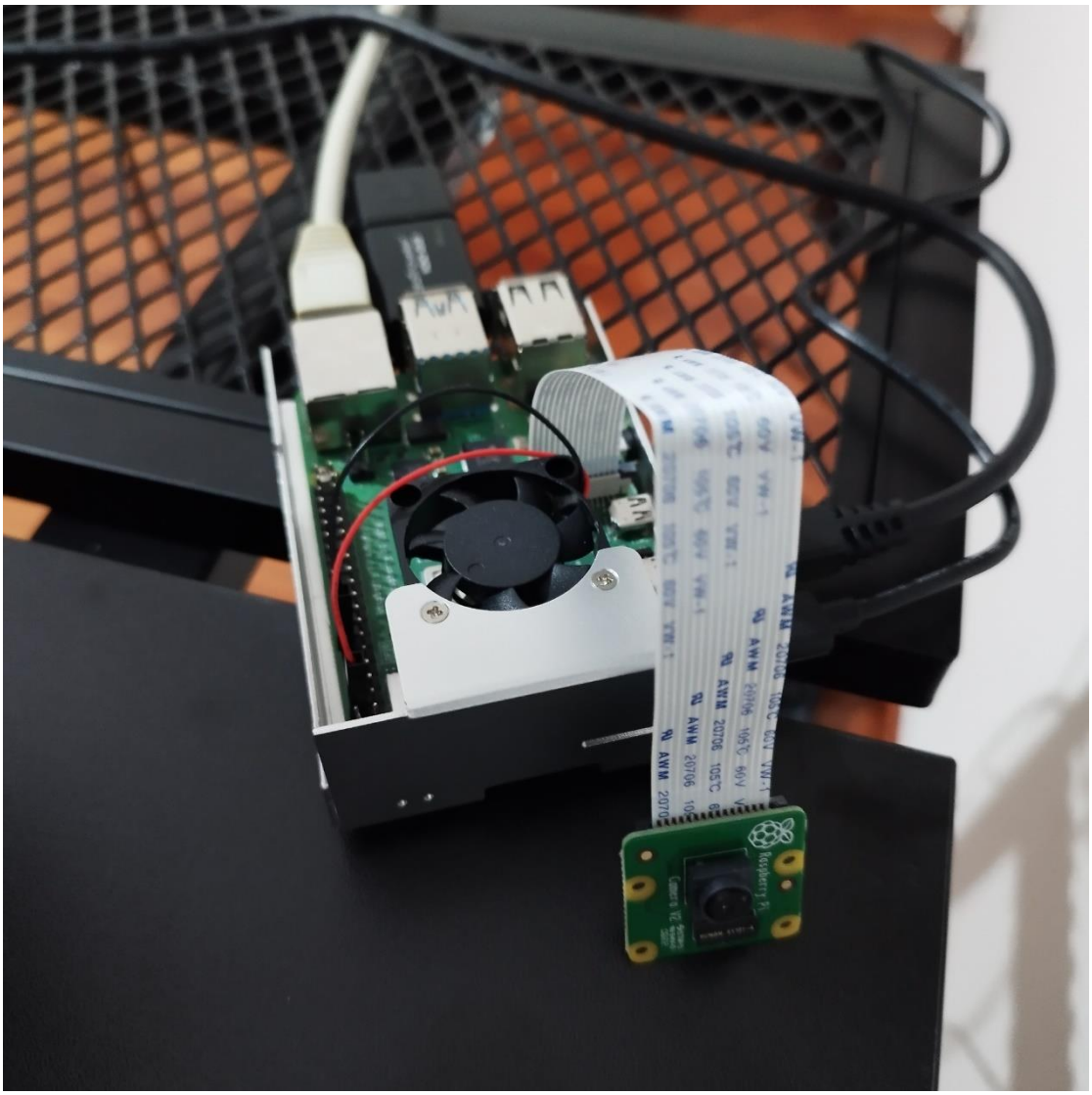

Figura 3: Raspberry pi 4 com câmara v2.0.

O raspberry tem uma placa de rede imbutida que é usada para aceder à internet e comunicar com o servidor. Na figura é possível observar a câmera conectada através da tira de comunicação à entrada designada do raspberry e uma ventoinha para arrefecimento do CPU em uso contínuo. A ventoinha foi conectada através do VCC e GND de 5 V da placa.

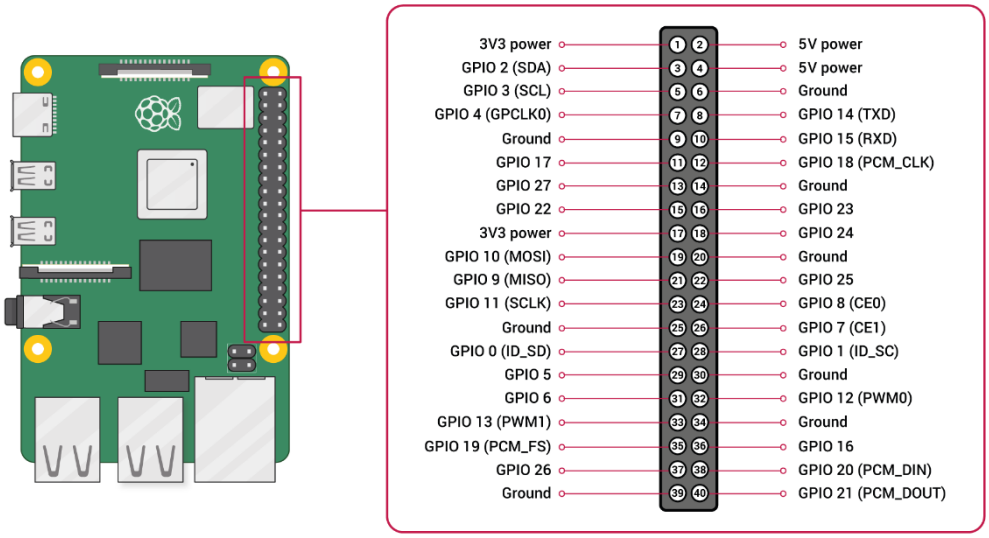

Figura 4: Arquitetura do raspberry pi 4 [23].

A figura 4 ilustra a arquitectura do raspberry, existem vários tipos de pinos para associar dispositivos externos. Uma das possibilidades era fazer a ligação apenas no raspberry mas, essa opção foi descartada porque os sensores têm um nível de compatibilidade superior com arduino e não tem pinos naturais de comunicação analógica.

De todas as possibilidades, optou-se por este tipo de configuração para permitir que a implementação da obtenção das imagens fosse desacoplada da obtenção dos dados dos sensores e pelo descrito anteriormente.

3.3.3 Visão global da solução

A solução é composta por várias fases para atingir o objectivo final da solução de visão computacional para deteção da poluição no ar. A primeira fase foi descrita anteriormente e representa a instalação da estrutura física. A segunda fase é a aquisição de dados e treino do modelo de visão computacional. A terceira fase é implementação do backend, a quarta fase é implementação do frontend e a quinta e ultima fase é a avaliação/testes da solução.

O seguinte diagrama permite compreender de forma simplificada todo o fluxo criado tendo em conta os diversos componentes e fases.

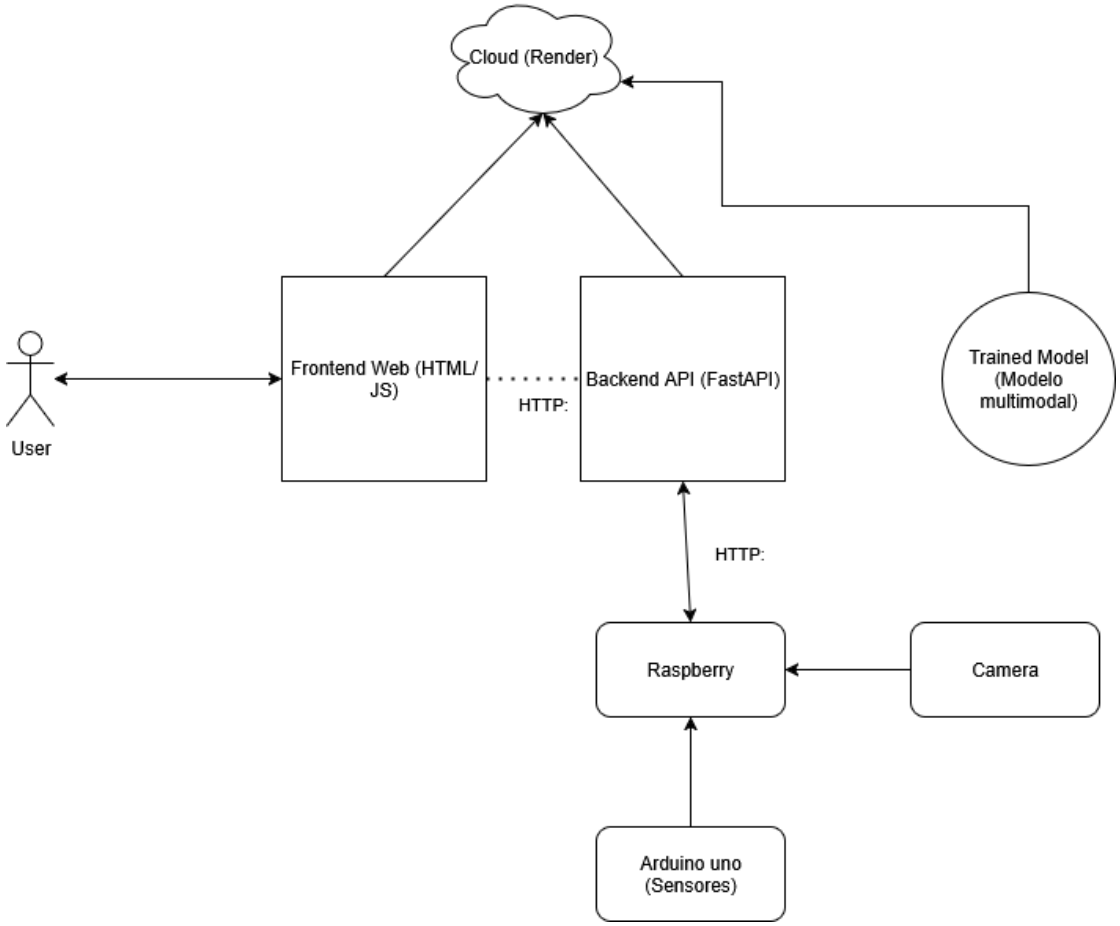

Figura 5: Diagrama geral da solução.

Optou-se por uma solução reduzida ao nível de software, ainda que de acordo com boas práticas de desenvolvimento, para validação do conceito e foco na caracterização efetuada pelo modelo dos dados obtidos.

O que se entende por boas práticas significa atribuir apenas uma responsabilidade a cada classe, criar interfaces para abstração do conteúdo de cada classe, baixo acoplamento e alta coesão e documentação.

Para perceber melhor o que foi implementado e, de acordo com os princípios, de arquitectura de software, realizou-se vários diagramas do sistema.

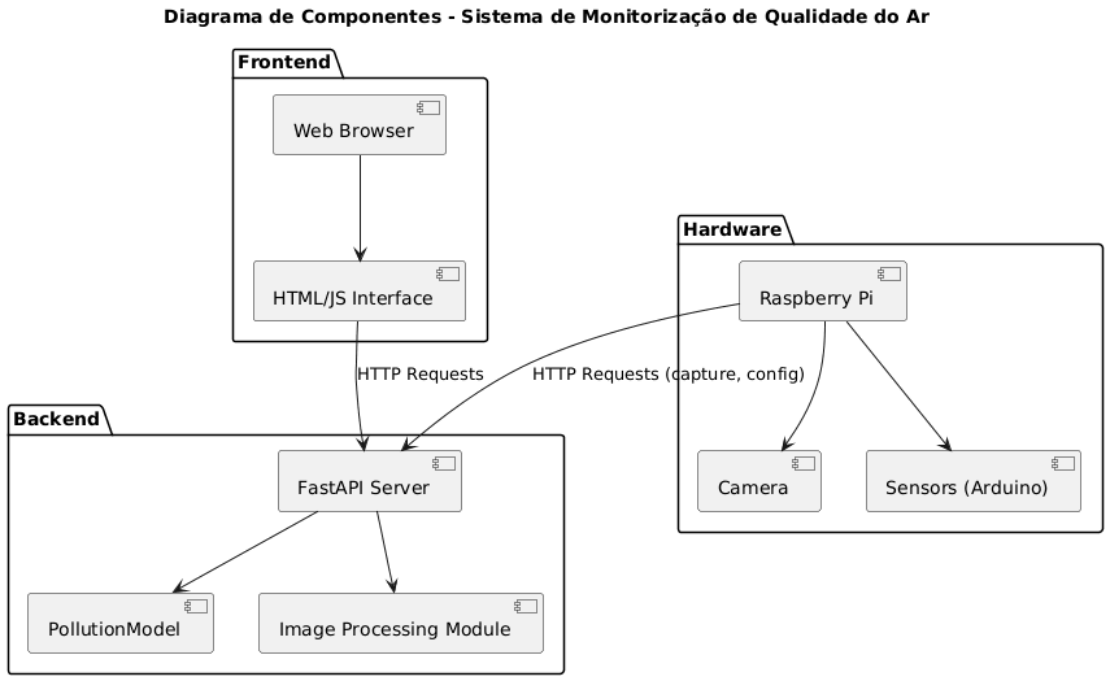

Figura 6: Diagrama de componentes.

O diagrama de componentes da figura permite compreender as interações entre os diferentes componentes do sistema. O PollutionModel e o ImageProcessingModule estão separados para representar duas ações diferentes. No entanto, na nossa arquitectura, fazem parte do componente modulo de visão computacional, o Image Processing faz o processamento das fotos capturadas e prepara para o formato adequado. O Pollution Model infere sobre a imagem e envia o resultado para a API.

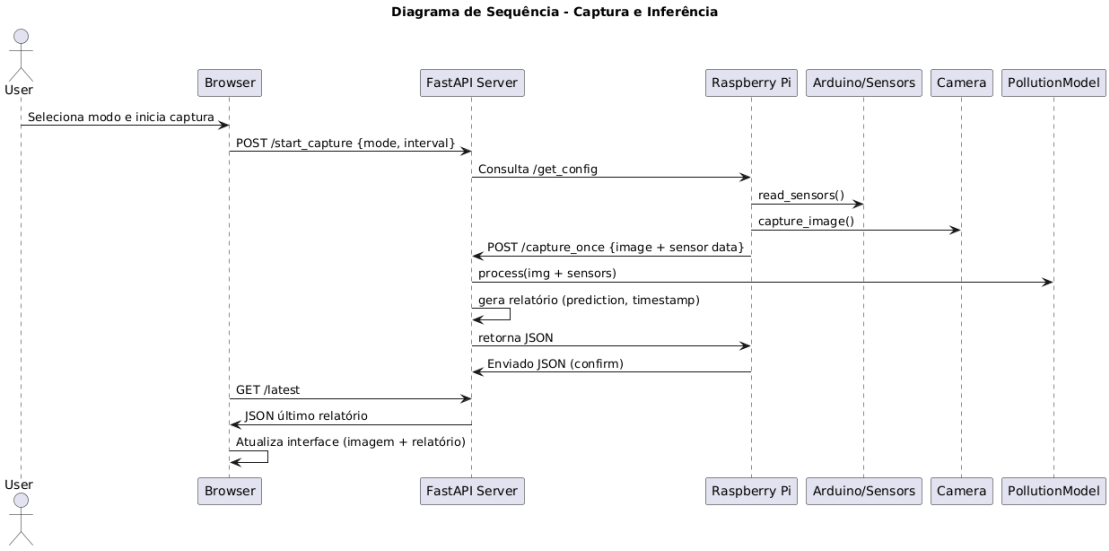

Figura 7: Diagrama de sequência, caso de uso captura e inferência.

O diagrama de sequência para um caso de uso (captura multimodal) identifica as interações entre cada classe e foi pensado para reduzir a complexidade de sistemas com várias camadas

(onion layer). A escolha de um sistema de IoT e Edge Computing foi feita pela simplicidade do objectivo em concreto. O foco é no modelo de visão computacional e não no software que envolve o sistema.

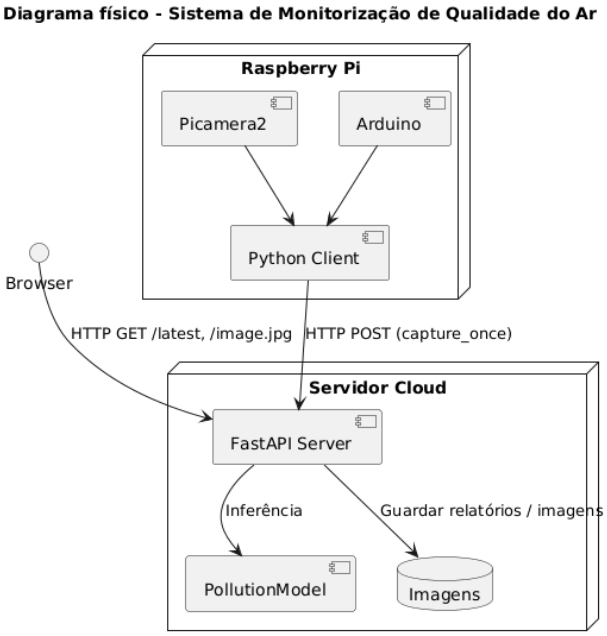

Figura 8: Diagrama físico do sistema.

No diagrama físico, representou-se a estrutura física de comunicação entre os blocos servidor e edge computer. Quanto ao sistema de armazenamento das imagens e relatórios escolheu-se guardar os ficheiros na própria cloud. É uma solução rápida mas, que com uma possível expansão para vários sistemas em combinação poderá ser necessário uma persistência mais tradicional (base de dados).

O projecto desenvolvido consiste em obter dados, treinar um modelo, montar um sistema incorporado de sensores ambientais e câmera, capturar em tempo real, processar a informação por visão computacional, aferir sobre os resultados e comunicar ao utilizador. Todo o sistema é representativo do conceito IoT e faz uso das melhores práticas fornecendo aos utilizadores fácil acesso ao resultado final por intermédio de conectividade web e de um cloud server para qualquer dispositivo.

3.3.4 Aquisição de dados e agrupamento

A obtenção de dados para treinar o modelo representou uma parte desafiante do projecto. Para treinar um modelo de visão computacional é necessário um grande número de dados. Tal como referido, optou-se por uma abordagem multimodal onde o modelo é treinado através de dados dos sensores e imagens para aferir sobre a qualidade do ar. O objectivo desta

abordagem multimodal é poder fornecer ao modelo, dados de maior qualidade para treino. Uma vez que, por vezes, a imagem não consegue ser representativa da qualidade do ar por causa de oscilações de ppm das moléculas. Tal como referido, este projecto diferencia-se dos estudos analisados por esta dualidade e porque o utilizador vai ter a possibilidade de usar apenas imagem ou imagem e dados para aferir sobre a qualidade do ar.

Para obter dados confiáveis utilizou-se o próprio sistema e capturou-se a concentração em ppm de CO, CO2 e PM2.5. Estas moléculas representam, de uma forma geral, os parâmetros que podem revelar a qualidade do ar. Associado às moléculas capturou-se a imagem no momento de análise do ar. Os dados foram guardados num ficheiro csv (exemplo em anexo) onde cada imagem tem dados associados e vice-versa. Para facilitar e catalogar cada imagem foi avaliada a qualidade do ar através de intervalos de valor da concentração de PM2.5.

Air Quality Index (AQI) Values	Levels of Health Concern
When the AQI is in this range:	...air quality conditions are:
0-50	Good
51-100	Moderate
101-150	Unhealthy for Sensitive Groups
151-200	Unhealthy
201-300	Very Unhealthy
310-500	Hazardous

Figura 9: Diagrama do índice de qualidade do ar [22].

A figura 9 exemplifica os intervalos de qualidade do ar e a sua designação. Utilizou-se esses intervalos para efetuar o agrupamento da cada imagem e a [22]capacidade do modelo se adaptar e treinar de acordo com os agrupamentos.

Uma vez que, o ar do local, onde o projecto foi desenvolvido, é tipicamente de boa qualidade, a base de dados adquirida foi unidimensional e de forte prevalência em boa qualidade de ar (figura 10).

Figura 10: Exemplo imagem capturada na zona da Maia, Portugal.

A homogeneidade dos dados faz com que não seja suficiente para um treino adequado do modelo. Para criar uma forte heterogeneidade nos dados adquiridos optou-se por capturar, através de webcams e arquivos, imagens de vários locais no mundo cuja qualidade do ar é fraca. A essas imagens associou-se, através do The World Air Quality Index Project, os índices de CO, CO2 e PM2.5, dos locais das fotos, no momento em que foram tiradas.

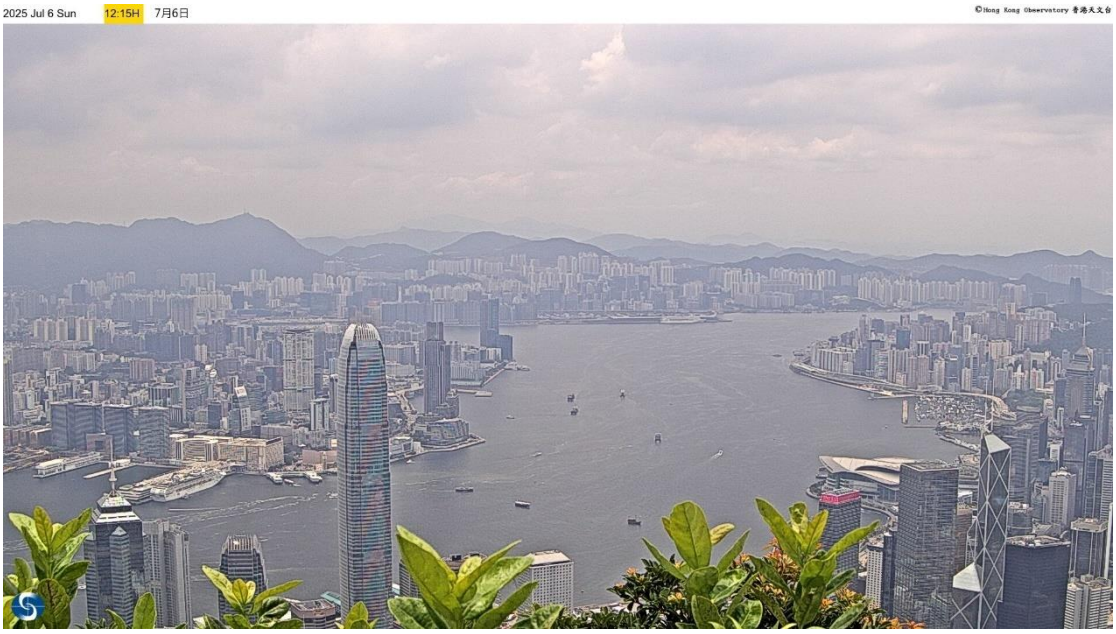

Figure 11: Cidade de Beijing, China, capturado por webcam (WebcamLive 2025).

Desta forma, foi possível ter uma base de dados de mais de mil imagens com dados associados, agrupadas por qualidade do ar e com heterogeneidade, para treinar o modelo.

3.3.5 Pré-processamento e treino do modelo

O pré-processamento de dados é uma etapa crítica para o sucesso de qualquer modelo de machine learning e assegura que os dados de entrada estão num formato adequado. Permite ainda que, os dados sejam consistentes e relevantes para o modelo aprender padrões significativos.

Para preparar os dados, antes treinar o modelo, ponderou-se várias práticas comuns de pré-processamento. Optou-se por efetuar normalização através da aplicação de Standard Scaling (transformação z-score), que permitirá colocar todas as variáveis na mesma escala dimensional. Esta abordagem é particularmente importante dado que os sensores operam com unidades de medida distintas (µg/m³ para partículas inaláveis e ppm para gases), com amplitudes de valores significativamente diferentes.

A arquitetura do modelo foi escolhida para equilibrar a precisão da deteção com a eficiência computacional, o que permite, executar o modelo em hardware de baixo custo e limitado como o raspberry pi. Utilizou-se uma sequência baseada em transformações compatíveis com arquitecturas CNN modernas. As imagens serão redimensionadas para 224×224 pixels, formato standard para redes pré-treinadas como a ResNet. Incluiu-se conversão obrigatória para espaço de cores RGB para garantir consistência no número de canais. Tudo isto são procedimentos habituais numa rede ResNet porque criam uma normalidade e padrão entre as imagens, ao nível de coloração, brilho e tamanho. Permite aumentar a eficiência computacional, padronizar o processo e é obrigatório por causa da arquitetura ResNet.

Para a codificação das labels de qualidade do ar, planeou-se utilizar codificação ordinal através de Label Encoding, o que transforma as categorias qualitativas em representações numéricas sequenciais. Esta abordagem mantém a relação ordinal inerente aos níveis de qualidade do ar e utilizou-se o agrupamento realizado das imagens obtidas.

A divisão do dataset será efetuada mediante amostragem estratificada, para garantir que a distribuição de classes seja preservada em todos os subconjuntos (treino, validação e teste).

Quanto à técnica de treino e dos seus parâmetros escolheu-se a opção de fusão tardia (late fusion). Esta abordagem permite que cada modalidade (imagem e sensores) seja processada por arquiteturas otimizadas para a sua natureza específica, preservando a integridade da informação antes da combinação dos dados. Efetivamente, os parâmetros são processados independentes uns dos outros e combinados só nas camadas finais da rede.

A seleção do otimizador Adam (Adaptive Moment Estimation) com learning rate de 0.001 baseou-se na sua robustez empírica para problemas de classificação multimodal. Adam adapta-se, automaticamente, às taxas de aprendizagem por parâmetro. Isto é particularmente útil em arquiteturas com componentes de natureza diferente (CNN pré-treinada e MLP para sensores). Descartou-se outras opções como Adagrad que é menos eficaz em problemas heterogéneos e AdamW que, embora seja superior a regularizar, é uma vantagem marginal em problemas de dimensão moderada.

A Cross-Entropy loss mantém-se a escolha padrão para problemas de classificação multiclasse, maximizando a divergência entre distribuições de probabilidade das classes.

Em termos de épocas, escolheu-se 10 épocas como o ponto intermédio entre underfitting e overfitting.

3.3.6 Validação e testes em tempo real

O modelo criado é avaliado pelo f-score, recall, accuracy, matriz de confusão e UAC para todos os parâmetros. O backend desenvolvido em FastAPI vai ser validado manualmente pelo uso próprio do sistema. Os endpoints que servem de comunicação entre os dados obtidos em tempo real, o modelo treinado e os relatórios gerados e a comunicação dos dados por html ao utilizador final também serão validados por teste do utilizador manual.

Para o UI optou-se por uma simples página html capaz de permitir mostrar os dados obtidos do sensor, a ultima foto criada e ainda, um relatório de avaliação da qualidade do ar através do modelo treinado.

Existem duas opções de utilização, a primeira é capturar imagem e dados para aferir a qualidade do ar, a segunda é capturar apenas imagem e deixar o modelo treinado aferir a qualidade do ar e valores para as concentrações de CO, CO2 e PM2.5.

Qualquer dispositivo pode aceder à rede interligada ao Pi e monitorizar o sistema através de browser. Torna-se possível validar através de logs no raspberry pi e no server sobre o estado e funcionamento dos componentes.

4 Implementação da Solução

A solução proposta foi implementada, tendo como base, um modelo de visão computacional multimodal, capaz de integrar simultaneamente informação proveniente de imagens captadas por câmara e dados recolhidos por sensores ambientais. O modelo combina uma rede neuronal convolucional (ResNet-18, pré-treinada em ImageNet) para extração de características visuais e uma rede densa dedicada ao processamento dos valores dos sensores (PM2.5, CO, CO_2). As representações obtidas em cada ramo são posteriormente concatenadas e processadas por uma camada totalmente ligada, resultando na classificação final da qualidade do ar.

O processo de treino foi realizado em PyTorch, utilizando como conjunto de dados um ficheiro CSV com as leituras dos sensores e uma pasta de imagens associadas. O pré-processamento incluiu normalização dos dados dos sensores com StandardScaler e codificação das classes de qualidade do ar com LabelEncoder. Para as imagens, aplicou-se redimensionamento e conversão para tensores. O treino decorreu durante 10 épocas com o otimizador Adam, função de perda cross-entropy e batch size de 16.

O sistema foi preparado para correr tanto em ambientes de alto desempenho (GPU) como em dispositivos mais modestos, como o Raspberry Pi, que desempenhará o papel de coletor de imagens e dados de sensores.

Para garantir a integração da solução com uma interface acessível, foi desenvolvido um servidor FastAPI, responsável por disponibilizar uma interface web (via HTML e JavaScript) acessível através de qualquer navegador.

4.1 Descrição da implementação

O desenho do sistema foi cumprido tal como planeado. Primeiro começou-se por montar todo o hardware e realizar a configuraçao.

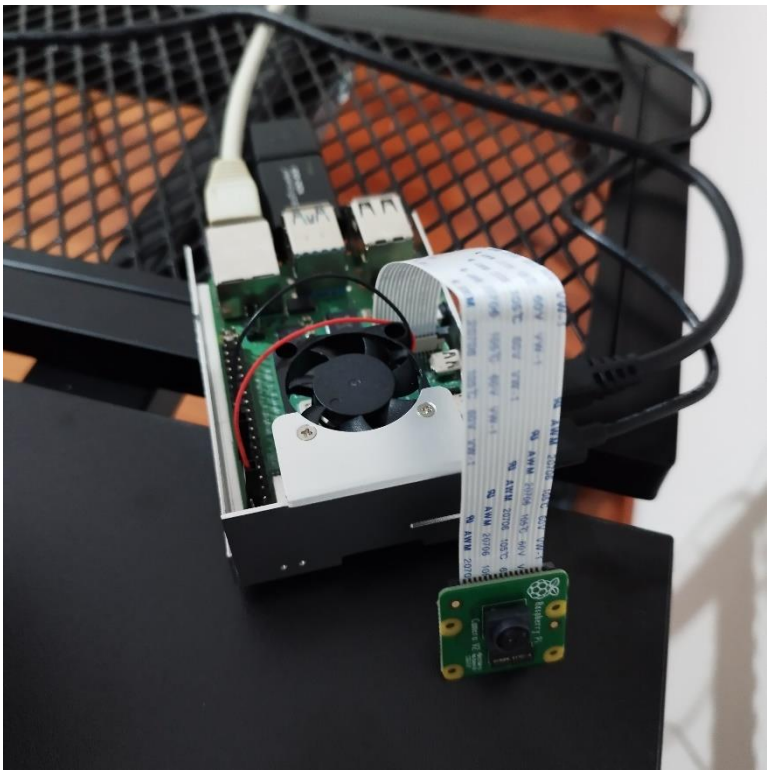

Figura 12: Raspberry pi 4 com câmera v2.0.

Nas figuras é possível verificar o sistema completo com o raspberry pi, câmera, conexão usb ao arduino. O arduino contêm os 4 sensores ligados em paralelo à alimentaçao de 5 V e os sensores atómicos às entradas analógicas e os sensores ambiente às entradas digitais.

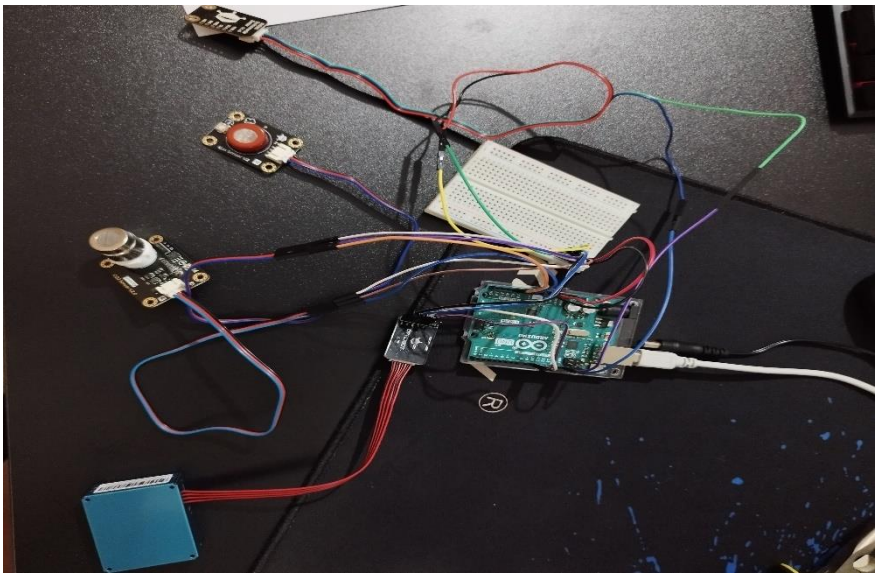

Figura 13: Instalação dos sensores e arduino uno.

A configuração dos sensores foi realizada de acordo com as especificaçoes do fabricante, a calibração do sensor de CO implicou obter o valor de Ro que corresponde ao valor padrão (ar ambiente puro). Utilizou-se o valor de 0.5 ppm de CO obtido através de registos metereológicos, para a cidade da Maia, no dia da calibração. O valor de Ro foi de 205 (adimensional).

Para a medição do CO2, o sensor requer aquecer durante 48 horas e efetuar uma leitura num local com ar limpo. O valor obtido dessa leitura foi de 0.405, calculado através da voltagem do sensor a dividir por 8.5, de acordo com as instruções do fabricante.

O SEN0233 e o SEN0248 não necessitam de calibração mas, requerem o nível do mar do local onde estão a ser utilizados. Para a cidade da Maia esse valor é 105 metros.

Após calibrados, combinaram-se as funcionalidades no mesmo loop para ser possível obter toda a informaçao no instante desejado de leitura.

```
// -------- MG-811 ------------
float volts = MGRead(MG_PIN);
int co2ppm = MGGetPercentage(volts, CO2Curve);
Serial.print("MG-811 | Voltage: "); Serial.print(volts); Serial.print(" V | CO2: ");
if(co2ppm == -1) Serial.print("<400"); else Serial.print(co2ppm);
Serial.print(" ppm | Alarm: "); Serial.println(digitalRead(BOOL_PIN) ? "OFF" : "ON");

// -------- MQ-7 -------------
int analog_value = analogRead(MQ7_PIN);
float sensor_voltage = (analog_value / 1023.0) * 5.0;
float Rs = (5.0 - sensor_voltage) / sensor_voltage * RL;
float ratio = Rs / Ro;
float co_ppm = 100 * pow(ratio, -1.5);
Serial.print("MQ-7  | Analog: "); Serial.print(analog_value);
Serial.print(" | Rs: "); Serial.print(Rs); Serial.print(" Ω");
Serial.print(" | Ratio: "); Serial.print(ratio);
Serial.print(" | CO: "); Serial.print(co_ppm); Serial.println(" ppm | Alarm: OFF");

// -------- SEN0233 ------------
while (senSerial.available() >= 32) {
  for(int i=0; i<32; i++) bufferRTT[i] = senSerial.read();
  unsigned int CR1 = (bufferRTT[30] << 8) + bufferRTT[31];
  unsigned int CR2 = 0;
  for(int i=0;i<30;i++) CR2 += bufferRTT[i];

  if(CR1 == CR2) {
    PMS1   = (bufferRTT[10] << 8) + bufferRTT[11];
    PMS2_5 = (bufferRTT[12] << 8) + bufferRTT[13];
    PMS10  = (bufferRTT[14] << 8) + bufferRTT[15];
    TPS    = (bufferRTT[24] << 8) + bufferRTT[25];
    HDS    = (bufferRTT[26] << 8) + bufferRTT[27];
  } else {
    PMS1 = PMS2_5 = 0;
  }
}
Serial.println("-----------------SEN0233----------------------------");
Serial.print("Temp : "); Serial.println(TPS / 10.0);      // °C
Serial.print("Humidity: "); Serial.println(HDS / 10.0);  // %RH
Serial.print("PM1.0 : "); Serial.println(PMS1);
Serial.print("PM2.5 : "); Serial.println(PMS2_5);
Serial.print("PM10  : "); Serial.println(PMS10);

// -------- BME680 ------------
bme.startConvert();
delay(1000);
bme.update();
Serial.println("-----------------BME680----------------------------");
Serial.print("Temperature(C) : "); Serial.println(bme.readTemperature() / 100.0, 2);
Serial.print("Pressure(Pa)    : "); Serial.println(bme.readPressure());
Serial.print("Humidity(%rh)   : "); Serial.println(bme.readHumidity() / 1000.0, 2);
Serial.print("Gas(ohm)       : "); Serial.println(bme.readGasResistance());
Serial.print("Altitude(m)    : "); Serial.println(bme.readAltitude());
#ifdef CALIBRATE_PRESSURE
  Serial.print("Calibrated Altitude(m): "); Serial.println(bme.readCalibratedAltitude(seaLevel));
#endif

delay(10000); // main loop delay
```

Figura 14: Excerto de código com configuração dos sensores no arduino IDE.

No excerto de código, é possível verificar que foi realizada uma conversão da voltagem lida dos sensores analógicos para converter o valor para ppm. Os digitais já têm essa conversão interna.

```
-----------------BME680----------------------------
Temperature(C) : 27.67
Pressure(Pa)    : 101242.00
Humidity(%rh)   : 50.61
Gas(ohm)       : 44102.00
Altitude(m)    : 21.39
---------------------------------------------------
MG-811 | Voltage: 2.72 V | CO2: 5296 ppm | Alarm: OFF
MQ-7  | Analog: 477 | Rs: 11446.54 Ω | Ratio: 55.84 | CO: 0.24 ppm | Alarm: OFF
-----------------SEN0233---------------------------
Temp : 21.80
Humidity: 56.30
PM1.0 : 0
PM2.5 : 0
```

Figura 15: Output da porta serial do arduino com valores obtidos dos sensores.

A figura 15 revela uma leitura dos sensores. Apesar de estarem corretamente calibrados a sua sensibilidade para valores inferiores a 5 ug/m3 de PM2.5 é relativamente baixa e o sensor de COVs é susceptivel ao aquecimento do próprio o que causa a inflação da temperatura lida.

No momento de leitura, a temperatura correta é do SEN0233 e coincide com a obtida pelo IPMA. Quanto ao nível de CO e CO2 estão com valores aceitáveis e concordantes com as medições desse dia de acordo com o AQICN.

Figura 16: Exemplo foto capturada para análise.

A figura 16 representa a captura efetuada no momento das figuras anteriores. De notar que, de uma forma subjectiva, podemos aferir que o ar parece puro e limpo o que valida os resultados obtidos. Com estes valores a imagem é catalogada como qualidade do ar boa.

O treino do modelo foi realizado através de mais de 1000 imagens e dados dos sensores idênticos aos exemplos anteriores.

Figure 17: Foto noturna capturada em Bombai, India (WebcamLive 2025).

Figura 18: Foto com smog em Beijing, China (WebcamLive 2025).

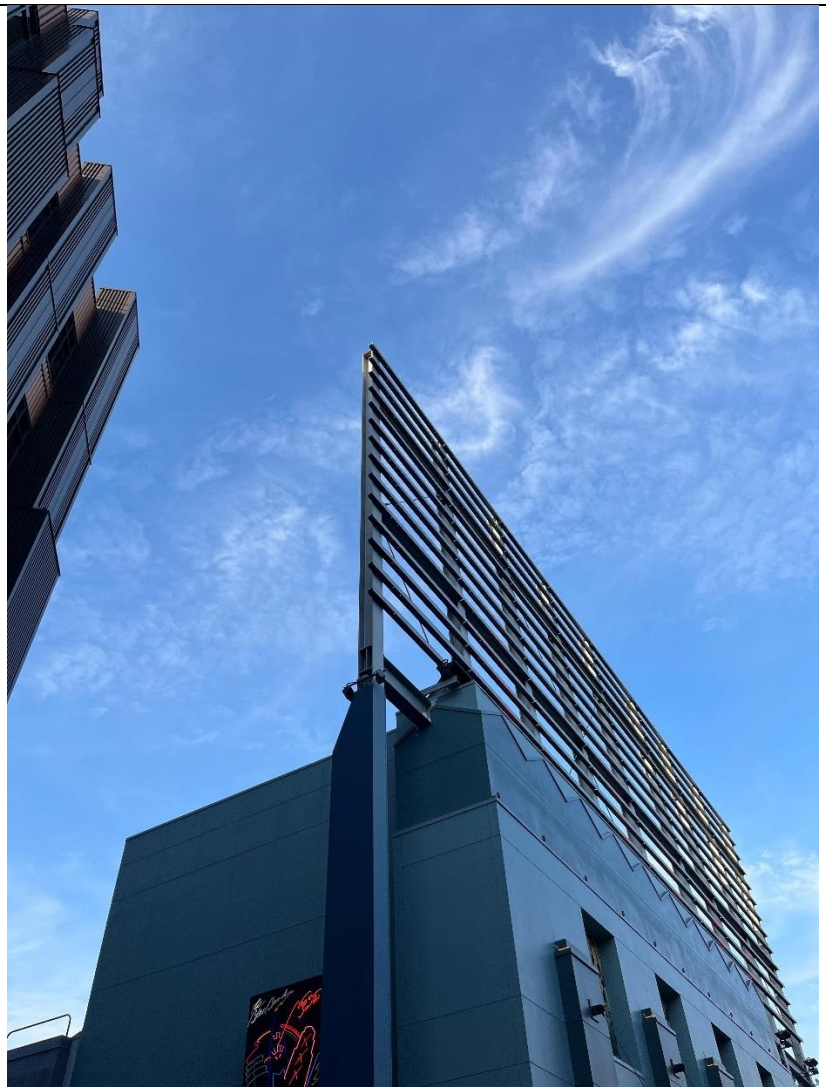

Figure 19: Foto na California, Estados Unidos (WebcamLive 2025).

Figure 20: Foto parque da cidade, Matosinhos, Portugal.

As figuras 17 a 21 representam exemplos que compõem o dataset utilizado para treinar o modelo. É possível verificar pelas figuras que existem situações diferentes. Essas situações podem afetar os resultados obtidos, o sol direto, noite, núvens, entre outros, podem induzir em erro o modelo. Por esse motivo, é importante ter diversidade de imagens. As capturas são armazenadas num ficheiro csv com informação da imagem e dados.

```
image_name,pm25,co,co2,qa
1712274700572,4.4,0.2,328,Good
1712274700813,5.7,0.3,358,Good
1712274701031,2.6,0.1,342,Good
1712274701271,6.7,0.2,323,Good
1712274701510,4.8,0.4,341,Good
```

Figura 21: Excerto do ficheiro csv utilizado com dados dos sensores e associação à imagem respectiva.

Após o pre-processamento dos dados e das imagens, o modelo foi criado com sucesso.

```python
# ========================
# 8. Modelo
# ========================
class PollutionModel(nn.Module):
    def __init__(self, num_classes, sensor_input_dim=3):
        super(PollutionModel, self).__init__()

        # CNN pré-treinada
        self.cnn = models.resnet18(pretrained=True)
        in_features = self.cnn.fc.in_features
        self.cnn.fc = nn.Identity()

        # Branch sensores
        self.sensor_fc = nn.Sequential(
            nn.Linear(sensor_input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 32),
            nn.ReLU()
        )

        # Combinação
        self.fc = nn.Sequential(
            nn.Linear(in_features + 32, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, img, sensors):
        img_feat = self.cnn(img)
        sensor_feat = self.sensor_fc(sensors)
        combined = torch.cat((img_feat, sensor_feat), dim=1)
        out = self.fc(combined)
        return out

num_classes = len(label_encoder.classes_)
model = PollutionModel(num_classes)

# ========================
# 9. Treino
# ========================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    for images, sensors, labels in train_loader:
        images, sensors, labels = images.to(device), sensors.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images, sensors)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{EPOCHS}], Loss: {avg_loss:.4f}")
```

Figura 22: Excerto código para criar e treinar o modelo de visão computacional.

No excerto de código de treino do modelo, visualizam-se algumas das configurações (restante código em anexo) que se optou por utilizar, nomeadamente, ReLU, combinação do CNN das imagens com sequencial dos sensores, ativou-se modo cuda para utilizar GPU para acelerar o treino, entre outros.

Cada época foi registada e podemos avaliar que as perdas e riscos diminuem com consequentes épocas.

Epoch [1/10], Loss: 0.8650 Epoch [2/10], Loss: 0.6651 Epoch [3/10], Loss: 0.4763 Epoch [4/10], Loss: 0.4097 Epoch [5/10], Loss: 0.2938 Epoch [6/10], Loss: 0.1813 Epoch [7/10], Loss: 0.1255 Epoch [8/10], Loss: 0.1401 Epoch [9/10], Loss: 0.1126 Epoch [10/10], Loss: 0.1000 Acurácia no conjunto de teste: 96.59%

Figura 23: Output dos ciclos de treino do modelo com respectivas perdas associadas.

Verifica-se que com o aumento das épocas o modelo gradualmente aumenta a acurácia e diminui as perdas até um valor de 0.1 de perdas e 96.59% de acurácia.

A avaliação do modelo foi conduzida com base no conjunto de testes reservados (30% dos dados originais, estratificado por classes).

```
=== Classification Report ===
                              precision    recall  f1-score   support

                        Good       1.00      0.91      0.96        47
                   Hazardous       1.00      1.00      1.00         8
                    Moderate       0.95      0.99      0.97        82
                   Unhealthy       0.96      1.00      0.98        26
Unhealthy for Sensitive Groups     0.92      0.92      0.92        13

                    accuracy                           0.97       176
                   macro avg       0.97      0.97      0.97       176
                weighted avg       0.97      0.97      0.97       176


=== Confusion Matrix ===
[[43  0  4  0  0]
 [ 0  8  0  0  0]
 [ 0  0 81  0  1]
 [ 0  0  0 26  0]
 [ 0  0  0  1 12]]
```

Figura 24: Métricas de avaliação do modelo criado e matriz de confusão.

O relatório realizado do modelo indica que existe uma taxa de sucesso muito elevada. É um modelo muito eficaz a detetar variações e oscilações nas imagens. Pela matriz de confusão verifica-se que, apenas obteve 6 falsos positivos em centenas de imagens e a grande maioria em imagens de qualidade do ar bom. Este fenómeno faz todo o sentido pois, é mais complicado detetar oscilações subtis nos valores da poluiçao de baixa concentraçao de PPM por apenas a iamgem.

O próximo passo foi criar a API através de FastAPI e preparar o raspberry pi para enviar os dados dos sensores e foto para o servidor. O server foi armazenado em cloud, atráves do aplicativo Render que permite fazer deploy sem custos e torna acessível a validação do projecto.

O modelo desenvolvido, o scaler e tudo associado à analise das imagens por parte da visão computacional foi adicionado ao server. Implementou-se vários ficheiros para criação do backend no server para comunicar com o frontend e com a obtenção de dados.

Para comunicar com o frontend foram desenvolvidos dois pedidos GET, um para enviar o relatório e outro para enviar a imagem.

```
41  @app.get("/")
42  async def index():
43      with open("index.html", "r", encoding="utf-8") as f:
44          return HTMLResponse(f.read())
45
46  @app.get("/latest")
47  async def latest_report():
48      if last_report is not None:
49          return JSONResponse(last_report)
50      elif os.path.exists("latest.jpg"):
51          return JSONResponse({
52              "message": "Último relatório disponível",
53              "timestamp": datetime.now().isoformat()
54          })
55      else:
56          return JSONResponse({"error": "Nenhum relatório disponível"})
57
58
59  @app.get("/image.jpg")
60  async def get_image():
61      if os.path.exists("latest.jpg"):
62          return FileResponse("latest.jpg", media_type="image/jpeg")
63      return JSONResponse({"error": "Nenhuma imagem disponível"})
64
```

Figure 25: Exemplo código para envio do relatório gerado.

O frontend utiliza 3 POST, um para iniciar a captura da imagem, um para parar a captura e outro para capturar apenas uma vez. Utilizou-se a captura única como base para ambos, ao iniciar captura continua e funciona em loop, a cada 60 segundos ativa a captura única e envia informação sobre os dados obtidos. O utilizador ao parar a captura interrompe o ciclo.

```python
82  @app.post("/capture_once")
83  async def capture_once(mode: str = Form("multimodal"), pm25: float = Form(...),
84                         co: float = Form(...), co2: float = Form(...), image: UploadFile = File(...)):
85      global last_report  # adiciona isso
86
87      # processa imagem
88      img_bytes = await image.read()
89      img = Image.open(io.BytesIO(img_bytes)).convert("RGB")
90      img_tensor = transform(img).unsqueeze(0).to(DEVICE)
91
92      # processa sensores
93      if mode == "multimodal":
94          sensor_data = torch.tensor([[pm25, co, co2]], dtype=torch.float32).to(DEVICE)
95      else:
96          sensor_data = torch.zeros((1, 3), dtype=torch.float32).to(DEVICE)
97
98      with torch.no_grad():
99          output = model(img_tensor, sensor_data)
100         pred = torch.argmax(output, dim=1).item()
101         label = CLASS_NAMES[pred]
102
103     # salva imagem
104     img.save("latest.jpg")
105
106     report = {
107         "prediction_id": pred,
108         "prediction_label": label,
109         "timestamp": datetime.now().isoformat(),
110         "image_url": "/image.jpg"
111     }
112
113     last_report = report  # salva para o /latest
114
115     return JSONResponse(report)
```

Figura 26: Excerto de código para captura da imagem e avaliação por parte do modelo.

Na figura 26 está explicito o uso do modo multimodal mas, o sistema verifica e infere sob as preferências através da verificação das configurações. Se o utilizador mudar o modo para apenas imagem a inferência é realizada sem valores dos sensores. A imagem obtida é classificada de acordo com o nível de poluição. A captura única gera o relatório que é enviado no formato JSON para o /latest do frontend e para o raspberry como uma forma de log.

```python
121 @app.get("/get_config")
122 async def get_config():
123     return JSONResponse({
124         "capture_active": capture_active,
125         "capture_mode": capture_mode,
126         "capture_interval": capture_interval
127     })
```

Figura 27: Excerto de código sobre configuração do modo de inferência.

O raspberry como funciona de forma continua a receber dados e tirar fotos necessita de informação sobre as modificações possíveis do utilizador. Para isso, é enviado a cada 5 segundos uma resposta com as configurações do utilizador, se a captura está activa e o modo de captura. Ao receber essa informação sobre os estados o raspberry vai atuar de acordo.

```python
10  SERVER_URL = "https://projecto-oju4.onrender.com"  # URL do backend
11  SLEEP_INTERVAL = 5  # tempo entre verificações do backend (segundos)
12  IMAGE_FOLDER = Path("images")
13  IMAGE_FOLDER.mkdir(exist_ok=True)
```

Figura 28: Excerto de código com tempo de verificação das configurações e url para conexão com o servidor..

A figura representa um excerto de código no raspberry que evidencia a comunicação com o servidor, os directórios de armazenamento das imagens e o tempo de verificação de estados ao backend.

```
46  def capture_image():
47      timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
48      img_path = IMAGE_FOLDER / f"pollution_{timestamp}.jpg"
49      picam2.capture_file(str(img_path))
50      return img_path
51
52  def send_to_server(pm25, co, co2, mode, img_path):
53      with open(img_path, "rb") as f:
54          files = {"image": f}
55          data = {"pm25": pm25, "co": co, "co2": co2, "mode": mode}
56          try:
57              res = requests.post(f"{SERVER_URL}/capture_once", data=data, files=files, timeout=30)
58              print("Enviado:", res.json())
59          except Exception as e:
60              print("Erro ao enviar:", e)
61
```

Figura 29: Envio dos dados dos sensores para o servidor.

Os sensores coletam informação a cada segundo, essa informação é armazenada para um POST de captura única. Após enviadas as informações dos últimos 30 segundos as mesmas são analisadas e é realizada a média dos valores para enviar. Em conjunto com a imagem, é criado um JSON e enviado ao servidor, como se verifica na figura.

Todo o desenho planeado foi cumprido, desde a instalação do hardware até ao deploy do servidor e criação do UI web. O código completo de todas as classes pode ser consultado em anexo. Também foi desenvolvido um documento tutorial para consulta online quanto ao uso, instalação e manutenção do sistema.

O html criado é muito simples mas, funciona perfeitamente e permite aos utilizadores ter uma experiência concreta e objectiva. A figura exemplifica a interface web desenvolvida sem imagem capturada e sem relatório gerado.

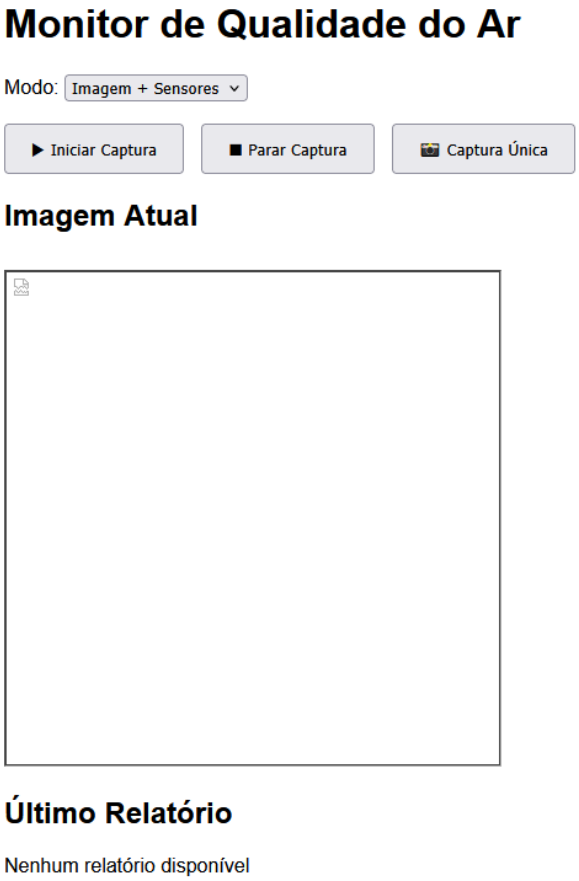

Figura 30: Interface web desenvolvida sem dados obtidos.

No desenvolvimento deste UI, optou-se por colocar 3 botões para capturar ou parar a captura e um de capturar de forma única. É possível escolher o modo de inferência (imagem ou imagem e dados) com um drop down menu. O pressionar dos botões ou alteração do modo envia uma resposta para o backend.

```javascript
1  async function startCapture() {
2    const mode = document.getElementById("mode").value;
3    await fetch("/start_capture", {
4      method: "POST",
5      body: new URLSearchParams({interval: 60, mode})
6    });
7    if (!intervalId) intervalId = setInterval(updateReport, 5000);
8  }
9
10 async function stopCapture() {
11   await fetch("/stop_capture", {method:"POST"});
12   if (intervalId) { clearInterval(intervalId); intervalId = null; }
13 }
14
15 async function captureOnce() {
16   const mode = document.getElementById("mode").value;
17   const pm25 = 0, co = 0, co2 = 0;
18   const formData = new FormData();
19   formData.append("mode", mode);
20   formData.append("pm25", pm25);
21   formData.append("co", co);
22   formData.append("co2", co2);
23   // imagem só no Raspberry
24   try {
25     const res = await fetch("/capture_once", {method:"POST", body: formData});
26     const data = await res.json();
27     document.getElementById("report").textContent = JSON.stringify(data, null, 2);
28     refreshImage();
29   } catch (err) {
30     console.error("Erro ao capturar:", err);
31   }
32 }
```

Figura 31: Implementação dos botões para captura.

O excerto de código mostra as 3 funções correspentes dos botões. Tal como planeado no desenho a gestão de erro e teste gera um log que é armazenado e as excepções são geridas e relativas apenas à conexão ou falha visto que não existem inputs especificos do utilizador.

Nos posts do ficheiro index verifica-se que a captura continua tem uma periocidade de 60 segundos e o modo escolhido influência no carácter de avaliação do modelo.

Durante o desenvolvimento e integração deste sistema, os testes realizados foram sobretudo de carácter exploratório e manual, uma vez que o projeto junta vários componentes distintos. Sensores ligados a um Arduino, um Raspberry Pi responsável pela recolha de dados e captura de imagens, um backend em FastAPI que processa a informação e corre o modelo de inferência, e finalmente um frontend que apresenta os resultados de forma acessível. Numa arquitetura distribuída e dependente de hardware, a prioridade passou por garantir que a comunicação entre estes elementos fluía de forma correta e que os relatórios eram gerados e visualizados no browser conforme esperado. Assim, os testes consistiram em verificar se os dados dos sensores eram lidos e enviados, se as imagens eram capturadas e processadas, se o backend recebia tudo e devolvia a classificação, e se o frontend refletia a informação em tempo real.

```python
@pytest.mark.asyncio
async def test_index_route_returns_html():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/")
    assert resp.status_code == 200
    assert "<html" in resp.text.lower()


@pytest.mark.asyncio
async def test_get_config_default_state():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/get_config")
    data = resp.json()
    assert "capture_active" in data
    assert "capture_mode" in data
    assert "capture_interval" in data


@pytest.mark.asyncio
async def test_latest_report_when_none():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/latest")
    data = resp.json()
    assert "error" in data or "message" in data
```

Figure 32: Exemplo testes unitários realizados ao backend.

Na figura 32, é possivel observar exemplos de testes unitários realizados ao backend. Testou-se todas as funcionalidades de comunicação, por exemplo, capacidade de detetar as alterações nas configurações ou obter o último relatório.

```python
Test_integration_e2e:
# tests/test_integration_e2e.py
import io
from PIL import Image
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_full_capture_flow():
    # 1. Verifica config inicial
    cfg = client.get("/get_config").json()
    assert cfg["capture_active"] is False

    # 2. Inicia captura
    client.post("/start_capture", data={"interval": 5, "mode": "multimodal"})

    # 3. Faz captura única
    img = Image.new("RGB", (224, 224), "blue")
    buf = io.BytesIO()
    img.save(buf, format="JPEG")
    buf.seek(0)
    files = {"image": ("fake.jpg", buf, "image/jpeg")}
    data = {"mode": "multimodal", "pm25": "12", "co": "0.5", "co2": "420"}
    report = client.post("/capture_once", files=files, data=data).json()
    assert "prediction_label" in report

    # 4. Último relatório deve estar disponível
    latest = client.get("/latest").json()
    assert latest["prediction_label"] == report["prediction_label"]
```

Figure 33: Exemplo teste e2e para testar captura do sistema.

Realizou-se testes de integração e e2e com o intuito de testar a resposta do raspberry pi com os sensores e o envio e captura de informação e de imagem pelo sistema de forma completa.

Na figura 34 é possivel observar a cobertura total de testes perante todas as classes e o sucesso dos mesmo.

```
PS                                                  \projecto-main> py -m pytest -v --maxfail=1 --disable-warnings
============================================= test session starts =============================================
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0
rootdir:                                                  \projecto-main
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.10.0, asyncio-1.1.0, cov-7.0.0
asyncio: mode=Mode.AUTO, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 12 items

tests\test_backend_api.py .......                                                                        [ 58%]
tests\test_frontend_api.py .                                                                             [ 66%]
tests\test_integration_e2e.py .                                                                          [ 75%]
tests\test_model.py ..                                                                                   [ 91%]
tests\test_raspberry_client.py .                                                                         [100%]

========================================== 12 passed, 6 warnings in 6.54s ==========================================
PS                                                  \projecto-main>
```

Figura 34: Relatório de execução de teste com percentagem de cobertura e aprovados/rejeitados.

Optou-se por não implementar uma bateria de testes automáticos complexos, sobretudo porque grande parte da lógica envolve dispositivos físicos, cuja simulação exigiria um esforço desproporcional em comparação com o valor acrescentado. Testar a lógica interna de funções que processam leituras de sensores ou capturam imagens não traria grande valor sem os dispositivos reais ligados, já que o comportamento esperado depende fortemente das condições ambientais e do hardware em uso.

Por exemplo, testar de forma automatizada a leitura do Arduino ou a captura de imagens pela câmara do Raspberry Pi exigiria mocks artificiais que, na prática, não refletiriam as condições reais de operação. Além disso, a fase de validação mais crítica é justamente a observação do sistema em funcionamento. Ver no terminal do Raspberry os dados médios a serem calculados, confirmar no backend que os relatórios são gerados com rótulos consistentes, e verificar no frontend a atualização visual desses relatórios. Esse ciclo de testes manuais mostrou-se suficiente para validar a robustez e a utilidade da solução.

4.2 Avaliação da solução

O sistema foi avaliado de acordo com a sua utilização base. Ou seja, realizou-se vários testes em condições diferentes. Durante a noite, com uma coluna de fumo, com proximidade de poluição conhecida, entre outros.

Figura 35: Exemplo de teste do sistema com imagem de ar limpo.

O primeiro exemplo capturado é concordante, o relatório gerado indica que a qualidade do ar é boa e os dados dos sensores confirmam isso mesmo. A imagem é limpa, o céu abrange grande parte da imagem e não existem nuvens.

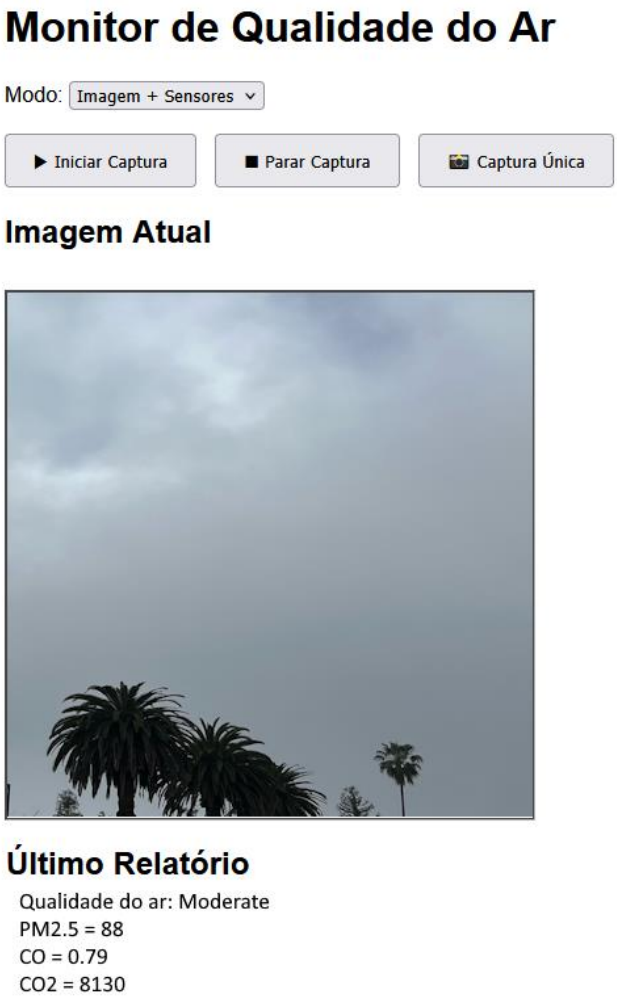

Figure 36: Foto com nuvens e céu cinzento.

Este segundo exemplo, também é concordante os valores dos sensores com a avaliação da imagem. No entanto, uma foto na mesma localização no momento seguinte mas, com o modo de imagem apenas ativo nota-se que o modelo tem maior dificuldade e classifica a foto como pouco saudável. Nesta situação, o modelo foi influenciado pelas nuvens da foto. Este facto mostra a diferença entre o uso do modelo com imagem ou com imagem e com dados. O facto de os dados serem um suporte para a tomada de decisão do modelo é evidente neste tipo de situações.

Figure 37: Foto com nuvens e céu cinzento mas, avaliação em modo imagem apenas.

No seguinte exemplo, foi testado um cenário noturno e o modelo conseguiu ficar em concordância, ambas as fotos são classificadas de ar bom.

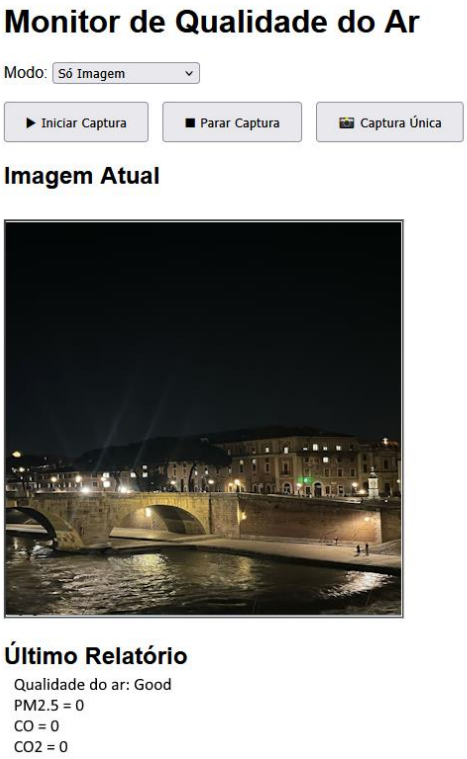

Figura 38: Foto noturna, modo imagem apenas.

O facto de os sensores serem um apoio ao modelo obrigam que, para altos níveis de eficâcia os sensores têm que captar para atingir alta fiabilidade. Esse problema é resolvido ao colocar os sensores perto das câmeras num conjunto de equipamento para ambos trabalharem em sintonia. Caso, a opção de ter sensores não seja viável então o modelo consegue detetar mas, com eficâcia reduzida.

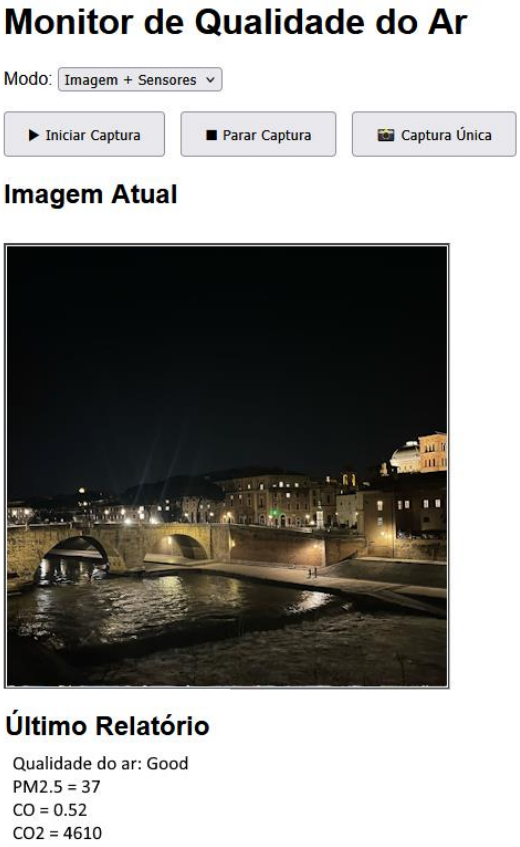

Figura 39: Foto noturna com modo imagem e dados.

No último exemplo de teste, a foto foi tirada durante um incêndio. Devido à distância do incêndio os sensores não conseguiram obter valores reais de dentro da coluna de fumo. O modelo responde de forma adequada, deteta a coluna de fumo e qualifica a qualidade do ar, com base nisso, como nociva. Isto significa que, o modelo tem a capacidade de filtrar e ser seletivo quando existe uma diferença entre a imagem e os valores dos sensores.

Figura 40: Foto com coluna de fumo.

O modelo responde de forma adequada, deteta a coluna de fumo e qualifica a qualidade do ar, com base nisso, como nociva. Isto significa que, o modelo tem a capacidade de filtrar e ser seletivo quando existe uma diferença entre a imagem e os valores dos sensores.

Em cenários noturnos ou poluição localizada distante, a importância de posicionar corretamente os sensores e câmeras é fundamental. Nestes casos, o modelo visual sozinho pode fornecer indicações, mas a acurácia aumenta quando há dados de sensores complementares.

Em termos de desempenho, o sistema mostrou-se rápido o suficiente para capturas periódicas, sem atrasos significativos, e a comunicação entre Raspberry Pi e servidor na cloud manteve-se estável, garantindo que os relatórios eram atualizados em tempo quase real. A avaliação também permitiu identificar algumas limitações. Tal como, a fraca precisão do modelo quando opera apenas com imagem em cenários com elementos visuais enganosos ou condições atmosféricas adversas. A dependência de sensores funcionais próximos à câmera neste tipo de situações é grande. Apesar dessas limitações, o comportamento do sistema foi consistente com os objetivos do projeto e demonstra que a solução é adequada para

monitoramento contínuo da qualidade do ar e capaz de fornecer informações tanto por via de sensores, quanto por via de análises de imagens.

Observou-se que a combinação de sensores com inferência visual melhora significativamente a fiabilidade da classificação, especialmente em situações ambíguas, como nuvens ou iluminação variável. O sistema também foi avaliado quanto à consistência temporal, capturas sucessivas em intervalos de 60 segundos mostraram estabilidade na leitura dos sensores e coerência nas previsões do modelo, indicando que a estratégia de média dos valores ao longo do período é eficaz.

Quando em comparação com técnicas habituais, por exemplo, sensores atmosféricos ou estações meteorologicas e até mesmo outros modulos de visão computacional o projecto desenvolvido tem a vantagem de combinar o melhor de ambos. Consegue ter a flexibilidade para detetar focos de poluição especificas nas imagens e consegue determinar níveis de poluentes não visiveis como fazem as estações de qualidade do ar. Ao juntarmos os dois métodos também criamos debilidades como falsos positivos e influencia da imagem ou dos sensores na decisão final.

Ao nível de custos o projecto é muito econômico e apresenta a vantagem das peças serem facilmente substituidas o que torna a manutenção fácil e rápida. A eficacia poderá subir consideravelmente se forem combinados vários sistemas iguais para cobrir uma área maior. Em especial, se forem desenvolvidos para se apoiarem uns aos outros na deteção o erro diminui.

5 Conclusões

O presente trabalho permitiu desenvolver e validar um sistema multimodal para monitorização da qualidade do ar. Combina leituras de sensores ambientais com análise de imagens capturadas em tempo real. O modelo treinado demonstrou elevada eficácia, atingindo uma acurácia de 96,59% e perdas reduzidas, com uma taxa mínima de falsos positivos concentrados sobretudo em imagens de ar de boa qualidade. Isto indica que, existe alguma dificuldade em detetar pequenas variações de poluição a partir de imagens só.

A avaliação do sistema em diferentes cenários, incluindo situações nocturnas, presença de nuvens e poluição localizada, como incêndios, demonstrou que a integração dos sensores com a análise visual aumenta significativamente a taxa de sucesso das classificações. A colocação estratégica dos sensores, próxima da câmera, revelou-se essencial para maximizar a precisão do modelo, sobretudo em condições ambíguas ou adversas. A consistência das capturas e previsões indica que a estratégia da média de valores é eficaz. Causa uma redução nas oscilações no que toca aos dados adquiridos.

Os objetivos iniciais foram alcançados com sucesso. Implementou-se um sistema capaz de capturar dados do ar e imagens. Criou-se um conjunto de dados multimodal para treinar e otimizar o modelo. O modelo de visão computacional integra a informação numa interface e disponibiliza um tutorial de configuração do sistema.

Para trabalhos futuros, recomenda-se explorar melhorias na robustez do modelo em cenários extremos, aumentar a cobertura espacial com mais sensores e câmeras, e investigar técnicas avançadas de fusão de dados multimodais.

5.1 Objetivos concretizados

Em termos de objectivos propostos e realizados, posso afirmar que todos foram cumpridos. É possível ver a implementação dos objectivos na secção 4 do relatório onde foi desenvolvido um sistema capaz de capturar leituras ambientais (CO_2, CO, PM2.5, temperatura, humidade, VOCs) e associá-las a imagens do local. Criou-se um conjunto de dados multimodal com exemplos reais para treino e validação de modelos de aprendizagem automática. Treinou-se e otimizou-se um modelo de visão computacional multimodal para identificação de fontes de poluição. Avaliou-se o desempenho do modelo em diferentes cenários, comparando-o com abordagens tradicionais baseadas apenas em sensores ou apenas em imagens. Realizou-se uma interface e um tutorial para configurar o sistema. Ao nível de interface web poderia estar um pouco melhor e mais polido mas, é perfeitamente funcional e escalável.

5.2 Limitações e trabalho futuro

O trabalho realizado foi muito exigente. Uma das limitações mais obvias é conseguir colectar uma grande quantidade de imagens associadas aos valores fidedignos da poluição. Penso que com um conjunto de treino muito maior, mais diversificado, seria possível aumentar a eficâcia do modelo.

Também considero que deveria ter desenvolvido um backend mais robusto a pensar em possíveis expansões do modelo para uma rede de modelos.

Para finalizar, o trabalho desenvolvido tem potencial para continuar e ser uma opção viável e talvez mais econômica do que as soluções atuais do mercado. No futuro, o desenvolvimento passa por melhorar os modelos de treino e expandir a rede de sensores e equipamentos para combinar os potenciais.

5.3 Apreciação final

O trabalho desenvolvido foi muito desafiante e gratificante. Devido à complexidade do projecto sinto que fui estimulado e cada passo concluido era uma vitória.

Ter a oportunidade de trabalhar com circuitos eletricos e equipamentos de eletronica também é muito interessante. A minha parte favorita do trabalho foi todo o processo de treino do modelo e dos circuitos embutidos.

6 Referências

[1] E. E. Agency, “Europe's air quality status 2023,” *EEA Report No 10/2023,* 2023.

[2] P. M. L. M. C. e. a. Kumar, “The rise of low-cost sensing for managing air pollution in cities,” *Environment International,* vol. 75, pp. 199-205, 2015.

[3] M. W. X. L. Y. M. Y. C. S. D. Xiaochu Wang, “Surveillance-image-based outdoor air quality monitoring,” *Environmental Science and ecotechnology,* vol. 18, nº 100319, 2024.

[4] X. Z. Y. S. T. A. S. A. A. A. A. G. W.-H. N. N. C. Maqsood Ahmed, “Low-cost video-based air quality estimation system using structured deep learning with selective state space modeling,” *Environment International,* vol. 199, nº 109496, 2025.

[5] P. a. D. W. Wohlschlegel, *Guideline for Development of a Quality Assurance Program: Volume IX—Visual Determination of Opacity Emissions from Stationary Sources,* November 1975.

[6] M. R. a. A. Stein, “Guidelines for Evaluation of Visible Emissions Certification, Field Procedures, Legal Aspects, and Background Material,” *EPA Publication No. EPA-340/1-75-007,* April 1975.

[7] Q. X. J. X. L. &. G. H. Zhang, “Deep Convolutional Neural Networks for Forest Fire Detection,” *Actas do Fórum Internacional sobre Gestão, Educação e Aplicação de Tecnologias de Informação.,* 2016.

[8] D. e. a. Rolnick, “Tackling Climate Change with Machine Learning,” *ACM Computing Surveys (CSUR),* p. 55, 2022.

[9] X. S. W. L. L. &. W. X. Li, “A Computer Vision-Based Method for Traffic-Related Air Pollution Estimation,” *Science of The Total Environment,* vol. 781, nº 146748, 2021.

[10] R. N. F. R. W. J. S. C. D. P. D. S. G. S. M. M. M. d. C. E. C. B. F.-F. J. O. M. B. L. d. S. &. C. C. L. Vasconcelos, "Fire Detection with Deep Learning: A Comprehensive Review," *Land,* vol. 13, nº 10, 2024.

[11] S. S. &. R. D. P. Kumar, "Global Operational Land Imager Landsat-8 Reflectance-Based Active Fire Detection Algorithm," *International Journal of Digital Earth,* vol. 13, nº 8, pp. 1-24, 2020.

[12] P. T. P. A. B. T. K. a. M. K. T. Bunpleng, "Smoke Segmentation Improvement Based on Fast Segment Anything," *n Proceedings of the 10th International Conference on Internet of Things, Big Data and Security (IoTBDS 2025),* pp. 129-140, 2025.

[13] ,. J. C. F. D. P. S. Z. C. L. Y. Q. T. K. X. S. C. Z. C. Q. F. R. L.-H. L. S.-H. B. C. S. H. Chaoning Zhang, "A Survey on Segment Anything Model (SAM): Vision Foundation Model Meets Prompt Engineering," *arXiv:2306.06211v4 [cs.CV],* 19 Outubro 2024.

[14] A. a. N. M. a. C. G. Dewangan, "FIgLib & SmokeyNet: Dataset and Deep Learning Model for Real-Time Wildland Fire Smoke Detection," *NeurIPS 2021 Workshop on Tackling Climate Change with Machine Learning,* 2021.

[15] M. &. C. M. &. M. L. &. K. H. &. R. A. &. J. D. Hardini, "Image-based Air Quality Prediction using Convolutional Neural Networks and Machine Learning," *Aptisi Transactions on Technopreneurship (ATT),* vol. 5, pp. 109-123, 2023.

[16] L. M. Laura Deveer, "Real-time air quality prediction using traffic videos and machine learning," *Transportation research part D: Transport and environment,* vol. 142, nº 104688, 2025.

[17] F. F. R. T. Qiang Zhang, "A deep learning and image-based model for air quality estimation," *Science of the total environment,* vol. 724, nº 138178, 2020.

[18] S. a. X. T. a. W. C. Q. a. L. G. a. Y. J. a. L. K. Xia, "AQMon: A Fine-grained Air Quality Monitoring System Based on UAV Images for Smart Cities," *Association for Computing Machinery,* vol. 20, nº 2, 2024.

[19] L. L. C. R. &. N. A. Bogdanffy, “Development of a Low-Cost Traffic and Air Quality Monitoring Internet of Things (IoT) System for Sustainable Urban and Environmental Management,” *Sustainability,* vol. 17, nº 11, 2025.

[20] Y. Y. S. H. a. J. X. Liang Zhao, “Design and implementation of a low-cost and multi-parameter indoor air quality detector based on IoT,” *International journal of computer applications in technology,* vol. 72, nº 4, 2023.

[21] F. F. R. T. Qiang Zhang, “A deep learning and image-based model for air quality estimation,” *Science of The Total Environment,* vol. 724, nº 138178, 2020.

[22] Arduino, “Arduino uno technical datasheet,” [Online]. Available: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf. [Acedido em 20 August 2025].

[23] Raspberry, “Raspberry pi 4 technical datasheet,” [Online]. Available: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html. [Acedido em 20 August 2025].

[24] “Air quality index basics.,” [Online]. Available: https://www.airnow.gov/aqi/aqi-basics/. [Acedido em 30 August 2025].

Anexo A Código dos sensores e raspberry pi.

Código dos sensores:

```
/************************* Libraries **************************/
#include <Wire.h>
#include <SPI.h>
#include "DFRobot_BME680_I2C.h"
#include <SoftwareSerial.h>

/************************* Pins ******************************/
#define MG_PIN       A0     // MG-811 analog output
#define BOOL_PIN      2     // MG-811 digital alarm
#define MQ7_PIN       A1     // MQ-7 analog pin

#define SEN0233_RX    10     // SoftwareSerial RX
#define SEN0233_TX    11     // SoftwareSerial TX

/************************* MG-811 / MQ-7 Config **************/
#define DC_GAIN 8.5
#define READ_SAMPLE_INTERVAL 50
#define READ_SAMPLE_TIMES    5
#define ZERO_POINT_VOLTAGE 0.405
#define REACTION_VOLTGAE   0.030

float Ro = 205.0;   // MQ-7 calibrated value
float CO2Curve[3]  = {2.602, ZERO_POINT_VOLTAGE, (REACTION_VOLTGAE/(2.602-3))};

#define RL 10000.0   // resistor de carga real do MQ-7 em ohms

/************************* SEN0233 Variables ******************/
SoftwareSerial senSerial(SEN0233_RX, SEN0233_TX);
unsigned char bufferRTT[32];
unsigned int PMS1=0, PMS2_5=0, PMS10=0, TPS=0, HDS=0;
```

```
/************************* BME680 Setup ***********************/
DFRobot_BME680_I2C bme(0x77);
float seaLevel;

/************************* Function Prototypes ***************/
float MGRead(int pin);
int MGGetPercentage(float volts, float *pcurve);

/************************* Setup ******************************/
void setup() {
  Serial.begin(115200);
  pinMode(BOOL_PIN, INPUT);
  digitalWrite(BOOL_PIN, HIGH);  // internal pull-up

  senSerial.begin(9600);

  Serial.println("Dor MG-811 + MQ-7 + SEN0233 + BME680 Started");
  Serial.println("MG-811 Warm-up required: Wait at least 2 hours");
  Serial.print("Using MQ-7 Ro value: ");
  Serial.println(Ro,2);

  // BME680 Init
  if(bme.begin()) {
    #ifdef CALIBRATE_PRESSURE
      bme.startConvert();
      delay(1000);
      bme.update();
      seaLevel = bme.readSeaLevel(104.0);
      Serial.print("BME680 seaLevel : ");
      Serial.println(seaLevel);
    #endif
  } else {
      Serial.println("BME680 not detected!");
  }
}

/************************* Loop *******************************/
void loop() {
  Serial.println("-----------------------------------------------");

  // -------- MG-811 ------------
  float volts = MGRead(MG_PIN);
  int co2ppm = MGGetPercentage(volts, CO2Curve);
  Serial.print("MG-811 | Voltage: "); Serial.print(volts); Serial.print(" V | CO2: ");
  if(co2ppm == -1) Serial.print("<400"); else Serial.print(co2ppm);
```

```
Serial.print(" ppm | Alarm: "); Serial.println(digitalRead(BOOL_PIN) ? "OFF" : "ON");

// -------- MQ-7 -------------
int analog_value = analogRead(MQ7_PIN);
float sensor_voltage = (analog_value / 1023.0) * 5.0;
float Rs = (5.0 - sensor_voltage) / sensor_voltage * RL;
float ratio = Rs / Ro;
float co_ppm = 100 * pow(ratio, -1.5);
Serial.print("MQ-7  | Analog: "); Serial.print(analog_value);
Serial.print(" | Rs: "); Serial.print(Rs); Serial.print(" Ω");
Serial.print(" | Ratio: "); Serial.print(ratio);
Serial.print(" | CO: "); Serial.print(co_ppm); Serial.println(" ppm | Alarm: OFF");

// -------- SEN0233 ------------
while (senSerial.available() >= 32) {
  for(int i=0; i<32; i++) bufferRTT[i] = senSerial.read();
  unsigned int CR1 = (bufferRTT[30] << 8) + bufferRTT[31];
  unsigned int CR2 = 0;
  for(int i=0;i<30;i++) CR2 += bufferRTT[i];

  if(CR1 == CR2) {
    PMS1   = (bufferRTT[10] << 8) + bufferRTT[11];
    PMS2_5 = (bufferRTT[12] << 8) + bufferRTT[13];
    PMS10  = (bufferRTT[14] << 8) + bufferRTT[15];
    TPS    = (bufferRTT[24] << 8) + bufferRTT[25];
    HDS    = (bufferRTT[26] << 8) + bufferRTT[27];
  } else {
    PMS1 = PMS2_5 = 0;
  }
}
Serial.println("-----------------SEN0233----------------------------");
Serial.print("Temp : "); Serial.println(TPS / 10.0);      // °C
Serial.print("Humidity: "); Serial.println(HDS / 10.0);  // %RH
Serial.print("PM1.0 : "); Serial.println(PMS1);
Serial.print("PM2.5 : "); Serial.println(PMS2_5);
Serial.print("PM10  : "); Serial.println(PMS10);

// -------- BME680 ------------
bme.startConvert();
delay(1000);
bme.update();
Serial.println("-----------------BME680---------------------------");
Serial.print("Temperature(C) : "); Serial.println(bme.readTemperature() / 100.0, 2);
Serial.print("Pressure(Pa)    : "); Serial.println(bme.readPressure());
Serial.print("Humidity(%rh)   : "); Serial.println(bme.readHumidity() / 1000.0, 2);
Serial.print("Gas(ohm)       : "); Serial.println(bme.readGasResistance());
```

```
  Serial.print("Altitude(m)    : "); Serial.println(bme.readAltitude());
  #ifdef CALIBRATE_PRESSURE
    Serial.print("Calibrated                    Altitude(m):                    ");
Serial.println(bme.readCalibratedAltitude(seaLevel));
  #endif

  delay(10000); // main loop delay
}

/**************** MG-811 Functions ******************/
float MGRead(int mg_pin) {
  float v=0;
  for(int i=0;i<READ_SAMPLE_TIMES;i++){
    v += analogRead(mg_pin);
    delay(READ_SAMPLE_INTERVAL);
  }
  v = (v/READ_SAMPLE_TIMES) * 5.0 / 1024.0;
  return v;
}

int MGGetPercentage(float volts, float *pcurve) {
  if ((volts / DC_GAIN) >= ZERO_POINT_VOLTAGE) return -1;
  return pow(10, ((volts / DC_GAIN) - pcurve[1]) / pcurve[2] + pcurve[0]);
}
```

Código raspberry:

```
import time, requests
from picamera2 import Picamera2
from datetime import datetime
from pathlib import Path
import serial

# =======================
# Configurações
# =======================
SERVER_URL = "https://projecto-oju4.onrender.com"  # URL do backend
SLEEP_INTERVAL = 5  # tempo entre verificações do backend (segundos)
IMAGE_FOLDER = Path("images")
IMAGE_FOLDER.mkdir(exist_ok=True)

# =======================
# Inicializa câmera e Arduino
# =======================
picam2 = Picamera2()
picam2.start()
```

```
arduino = serial.Serial("/dev/ttyACM0", 115200, timeout=1)
time.sleep(2)  # aguarda conexão Arduino

# =======================
# Funções utilitárias
# =======================
def read_sensors():
    arduino.write(b"read\n")
    line = arduino.readline().decode().strip()
    data = {"pm25":0, "co":0, "co2":0}  # defaults
    try:
        for sensor in ["pm25","co","co2"]:
            if sensor in line.lower():
                # extrai float do texto
                import re
                match = re.search(r"[-+]?\d*\.\d+|\d+", line)
                if match:
                    data[sensor] = float(match.group())
    except Exception as e:
        print("Erro parse sensor:", line, e)
    return data



def capture_image():
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    img_path = IMAGE_FOLDER / f"pollution_{timestamp}.jpg"
    picam2.capture_file(str(img_path))
    return img_path

def send_to_server(pm25, co, co2, mode, img_path):
    with open(img_path, "rb") as f:
        files = {"image": f}
        data = {"pm25": pm25, "co": co, "co2": co2, "mode": mode}
        try:
            res = requests.post(f"{SERVER_URL}/capture_once", data=data, files=files,
timeout=30)
            print("Enviado:", res.json())
        except Exception as e:
            print("Erro ao enviar:", e)

# =======================
# Loop principal
# =======================
while True:
    try:
```

```
        cfg = requests.get(f"{SERVER_URL}/get_config", timeout=10).json()
        if cfg["capture_active"]:
            interval = cfg["capture_interval"]
            mode = cfg["capture_mode"]

            # captura sensores durante o intervalo e tira média
            pm25_sum = co_sum = co2_sum = count = 0
            start_time = time.time()
            while time.time() - start_time < interval:
                s = read_sensors()
                pm25_sum += s.get("pm25", 0)
                co_sum += s.get("co", 0)
                co2_sum += s.get("co2", 0)
                count += 1
                time.sleep(1)

            pm25_avg = pm25_sum / max(count,1)
            co_avg = co_sum / max(count,1)
            co2_avg = co2_sum / max(count,1)

            img_path = capture_image()
            send_to_server(pm25_avg, co_avg, co2_avg, mode, img_path)
        else:
            time.sleep(SLEEP_INTERVAL)
    except Exception as e:
        print("Erro loop:", e)
        time.sleep(SLEEP_INTERVAL)
```

Código para ler Arduino:

```
import serial
import time
import re
from datetime import datetime

# Configurações da porta serial - IMPORTANTE: 115200 baud!
SERIAL_PORT = '/dev/ttyACM0'
BAUD_RATE = 115200  # ← ALTERADO para 115200!

def parse_arduino_data(raw_data):
    """
    Parseia os dados formatados do Arduino
    """
    data = {}

    try:
        # Procurar padrões nos dados
```

```
lines = raw_data.split('\n')

for line in lines:
    line = line.strip()

    # MG-811 CO2
    if 'MG-811 | Voltage:' in line:
        match = re.search(r'CO2:\s*([<\d]+)', line)
        if match:
            co2_str = match.group(1)
            if '<' in co2_str:
                data['co2'] = 400  # Valor mínimo
            else:
                data['co2'] = int(co2_str)

    # MQ-7 CO
    elif 'MQ-7  |' in line and 'CO:' in line:
        match = re.search(r'CO:\s*([\d.]+)', line)
        if match:
            data['co'] = float(match.group(1))

    # SEN0233 - PM2.5
    elif 'PM2.5 :' in line:
        match = re.search(r'PM2\.5\s*:\s*(\d+)', line)
        if match:
            data['pm25'] = int(match.group(1))

    # SEN0233 - Temperature
    elif 'Temp :' in line:
        match = re.search(r'Temp\s*:\s*([\d.]+)', line)
        if match:
            data['temperature'] = float(match.group(1))

    # SEN0233 - Humidity
    elif 'Humidity:' in line:
        match = re.search(r'Humidity:\s*([\d.]+)', line)
        if match:
            data['humidity'] = float(match.group(1))

    # BME680 - Temperature
    elif 'Temperature(C) :' in line:
        match = re.search(r'Temperature\(C\)\s*:\s*([\d.]+)', line)
        if match:
            data['bme_temperature'] = float(match.group(1))

    # BME680 - Pressure
```

```
        elif 'Pressure(Pa)    :' in line:
            match = re.search(r'Pressure\(Pa\)\s*:\s*([\d.]+)', line)
            if match:
                data['pressure'] = float(match.group(1))

        # BME680 - Gas
        elif 'Gas(ohm)       :' in line:
            match = re.search(r'Gas\(ohm\)\s*:\s*([\d.]+)', line)
            if match:
                data['gas_resistance'] = float(match.group(1))

    except Exception as e:
        print(f"Erro no parsing: {e}")

    return data

def read_arduino_data():
    """
    Lê dados do Arduino com baud rate correto
    """
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
        print(f"Conectado ao Arduino em {SERIAL_PORT} (115200 baud)")
        print("Aguardando dados...")
        print("-" * 60)

        # Dar tempo para Arduino estabilizar
        time.sleep(2)
        ser.reset_input_buffer()

        buffer = ""
        start_time = time.time()

        while time.time() - start_time < 15:  # 15 segundos de timeout
            if ser.in_waiting > 0:
                # Ler todos os dados disponíveis
                raw_bytes = ser.read(ser.in_waiting)

                try:
                    # Decodificar como UTF-8
                    new_data = raw_bytes.decode('utf-8', errors='replace')
                    buffer += new_data

                    # Se tiver uma seção completa (até BME680)
                    if "Altitude(m)" in buffer and "----------" in buffer:
                        print("Dados completos recebidos:")
```

```
                print(buffer)

                # Parsear dados
                parsed_data = parse_arduino_data(buffer)

                if parsed_data:
                    print("\nDados parseados:")
                    for key, value in parsed_data.items():
                        print(f"  {key}: {value}")

                    ser.close()
                    return parsed_data
                else:
                    print(" Não foi possível parsear dados")
                    buffer = ""  # Limpar buffer para próxima tentativa

            except UnicodeDecodeError:
                print(f" Bytes recebidos: {raw_bytes.hex()}")

        time.sleep(0.1)

    print(" Timeout - Dados incompletos recebidos:")
    print(buffer)
    ser.close()
    return None

except serial.SerialException as e:
    print(f" Erro de serial: {e}")
    return None
except KeyboardInterrupt:
    print("\n Leitura interrompida")
    if 'ser' in locals() and ser.is_open:
        ser.close()
    return None

def continuous_monitor():
    """
    Monitoramento contínuo dos dados
    """
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
        print(" Monitoramento contínuo iniciado (Ctrl+C para parar)")

        while True:
            if ser.in_waiting > 0:
                raw_bytes = ser.read(ser.in_waiting)
```

```
            try:
                data = raw_bytes.decode('utf-8', errors='replace')
                print(f" {datetime.now().strftime('%H:%M:%S')}")
                print(data)
                print("-" * 40)

                # Parsear rapidamente valores importantes
                lines = data.split('\n')
                for line in lines:
                    if 'CO2:' in line or 'CO:' in line or 'PM2.5' in line:
                        print(f" {line.strip()}")

            except UnicodeDecodeError:
                print(f" Bytes: {raw_bytes.hex()}")

        time.sleep(1)

    except KeyboardInterrupt:
        print("\nParando monitoramento...")
    finally:
        if 'ser' in locals() and ser.is_open:
            ser.close()

if __name__ == "__main__":
    print("=" * 70)
    print("LEITOR DE DADOS DO ARDUINO - COM BME680, MG-811, MQ-7, SEN0233")
    print("=" * 70)

    # Testar comunicação
    sensor_data = read_arduino_data()

    if sensor_data:
        print("\n Dados recebidos com sucesso!")
        print("Resumo dos valores:")
        print(f"CO₂: {sensor_data.get('co2', 'N/A')} ppm")
        print(f"CO: {sensor_data.get('co', 'N/A')} ppm")
        print(f"PM2.5: {sensor_data.get('pm25', 'N/A')} µg/m³")
        print(f"Temperatura: {sensor_data.get('temperature', 'N/A')}°C")
        print(f"Humidade: {sensor_data.get('humidity', 'N/A')}%")
    else:
        print("\n Não foi possível receber dados")

    # Oferecer monitoramento contínuo
    try:
        choice = input("\n Iniciar monitoramento contínuo? (s/n): ")
        if choice.lower() == 's':
```

```
        continuous_monitor()
    except:
        pass
```

Código para capturar foto da câmera:

```
from picamera2 import Picamera2
from time import sleep
from datetime import datetime
import os

def capture_pollution_image(filename_prefix="pollution"):
    """Captura imagem para o projeto de poluição"""

    # Criar diretório se não existir
    if not os.path.exists("images"):
        os.makedirs("images")

    # Gerar nome do arquivo com timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"images/{filename_prefix}_{timestamp}.jpg"

    # Capturar imagem
    picam2 = Picamera2()
    config = picam2.create_still_configuration()
    picam2.configure(config)
    picam2.start()
    sleep(2)  # Auto-ajuste
    picam2.capture_file(filename)
    picam2.stop()

    return filename

image_path = capture_pollution_image()
print(f"Imagem capturada: {image_path}")
```

Código para treinar o modelo:

```
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from torch.utils.data import Dataset, DataLoader
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms, models
```

```
from PIL import Image

# ========================
# CONFIGURAÇÕES
# ========================
CSV_PATH = "sensor_data.csv"  # ficheiro com dados
IMG_DIR = "../Documents/projetofimcurso/project/images"  # pasta com imagens
BATCH_SIZE = 16
EPOCHS = 10
LR = 0.001

# ========================
# 1. Carregar dataset
# ========================
df = pd.read_csv(CSV_PATH)

# Forçar conversão das colunas numéricas, substituir inválidos por NaN e remover linhas com
NaN
for col in ["pm25", "co", "co2"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df = df.dropna(subset=["pm25", "co", "co2", "qa"])  # garantir colunas limpas
df = df.reset_index(drop=True)

# ========================
# 2. Preparar labels (qa)
# ========================
label_encoder = LabelEncoder()
df["qa"] = label_encoder.fit_transform(df["qa"])

print("Classes encontradas:", list(label_encoder.classes_))

# ========================
# 3. Normalização dos sensores
# ========================
scaler = StandardScaler()
df[["pm25", "co", "co2"]] = scaler.fit_transform(df[["pm25", "co", "co2"]])

# ========================
# 4. Split train/val/test
# ========================
train_df, temp_df = train_test_split(df, test_size=0.3, stratify=df["qa"], random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.5, stratify=temp_df["qa"],
random_state=42)

print("Train classes:", train_df["qa"].unique())
```

```
print("Val classes:", val_df["qa"].unique())
print("Test classes:", test_df["qa"].unique())

# =======================
# 5. Dataset personalizado
# =======================
class PollutionDataset(Dataset):
    def __init__(self, dataframe, img_dir, transform=None):
        self.df = dataframe
        self.img_dir = img_dir
        self.transform = transform
        self.images = self.df["image_name"].values
        self.sensor_data = self.df[["pm25", "co", "co2"]].values.astype(np.float32)
        self.labels = self.df["qa"].values.astype(int)  # já são inteiros

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_name = str(self.images[idx]) + ".jpg"
        img_path = os.path.join(self.img_dir, img_name)

        image = Image.open(img_path).convert("RGB")
        if self.transform:
            image = self.transform(image)

        sensors = torch.tensor(self.sensor_data[idx])
        label = torch.tensor(self.labels[idx], dtype=torch.long)

        return image, sensors, label

# =======================
# 6. Transforms para imagens
# =======================
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# =======================
# 7. DataLoaders
# =======================
train_dataset = PollutionDataset(train_df, IMG_DIR, transform)
val_dataset = PollutionDataset(val_df, IMG_DIR, transform)
test_dataset = PollutionDataset(test_df, IMG_DIR, transform)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# =======================
# 8. Modelo
# =======================
class PollutionModel(nn.Module):
    def __init__(self, num_classes, sensor_input_dim=3):
        super(PollutionModel, self).__init__()

        # CNN pré-treinada
        self.cnn = models.resnet18(pretrained=True)
        in_features = self.cnn.fc.in_features
        self.cnn.fc = nn.Identity()  # remover fully connected final

        # Branch sensores
        self.sensor_fc = nn.Sequential(
            nn.Linear(sensor_input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 32),
            nn.ReLU()
        )

        # Combinação
        self.fc = nn.Sequential(
            nn.Linear(in_features + 32, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, img, sensors):
        img_feat = self.cnn(img)
        sensor_feat = self.sensor_fc(sensors)
        combined = torch.cat((img_feat, sensor_feat), dim=1)
        out = self.fc(combined)
        return out

num_classes = len(label_encoder.classes_)
model = PollutionModel(num_classes)

# =======================
# 9. Treino
# =======================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    for images, sensors, labels in train_loader:
        images, sensors, labels = images.to(device), sensors.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images, sensors)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{EPOCHS}], Loss: {avg_loss:.4f}")

# =======================
# 10. Validação final
# =======================
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, sensors, labels in test_loader:
        images, sensors, labels = images.to(device), sensors.to(device), labels.to(device)
        outputs = model(images, sensors)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Acurácia no conjunto de teste: {100 * correct / total:.2f}%")

# =======================
# 11. Guardar modelo
# =======================
torch.save(model.state_dict(), "pollution_model.pth")
print("Modelo treinado e guardado em pollution_model.pth")
```

Código para avaliar o modelo:

```
# evaluate.py
```

```
import torch
from torch.utils.data import DataLoader
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import os
import shutil

from dataset_and_model import (
    PollutionDataset, PollutionModel,
    label_encoder, test_df, IMG_DIR, transform
)

# ========================
# 1. Preparar dataset de teste
# ========================
test_dataset = PollutionDataset(test_df, IMG_DIR, transform)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# ========================
# 2. Carregar modelo
# ========================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_classes = len(label_encoder.classes_)

model = PollutionModel(num_classes)
model.load_state_dict(torch.load("pollution_model.pth", map_location=device))
model.to(device)
model.eval()

# ========================
# 3. Avaliação
# ========================
all_labels = []
all_preds = []
all_probs = []  # guardar probabilidades

with torch.no_grad():
    for images, sensors, labels in test_loader:
        images, sensors, labels = images.to(device), sensors.to(device), labels.to(device)
        outputs = model(images, sensors)

        probs = torch.softmax(outputs, dim=1)
        _, predicted = torch.max(probs.data, 1)
```

```
    all_labels.extend(labels.cpu().numpy())
    all_preds.extend(predicted.cpu().numpy())
    all_probs.extend(probs.cpu().numpy())

# =======================
# 4. Relatório e Confusion Matrix
# =======================
print("\n=== Classification Report ===")
print(classification_report(all_labels, all_preds, target_names=label_encoder.classes_))

cm = confusion_matrix(all_labels, all_preds)
print("\n=== Confusion Matrix ===")
print(cm)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=label_encoder.classes_,
        yticklabels=label_encoder.classes_)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

# =======================
# 5. Análise de Erros
# =======================
errors_idx = [i for i, (t, p) in enumerate(zip(all_labels, all_preds)) if t != p]

print(f"\nTotal de erros: {len(errors_idx)}")

# criar pasta para salvar erros
error_dir = "errors"
if os.path.exists(error_dir):
  shutil.rmtree(error_dir)
os.makedirs(error_dir, exist_ok=True)

for idx in errors_idx:
  row = test_df.iloc[idx]
  true_label = label_encoder.classes_[row["qa"]]
  pred_label = label_encoder.classes_[all_preds[idx]]

  img_name = str(row["image_name"]) + ".jpg"
  src = os.path.join(IMG_DIR, img_name)
  dst = os.path.join(error_dir, f"true_{true_label}_pred_{pred_label}_{img_name}")
```

```
    if os.path.exists(src):
        shutil.copy(src, dst)

print(f"Imagens de erros copiadas para pasta: {error_dir}")

# =======================
# 6. Curvas ROC/AUC (One-vs-All)
# =======================
from sklearn.preprocessing import label_binarize

y_true_bin = label_binarize(all_labels, classes=range(num_classes))
y_score = np.array(all_probs)

fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(num_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC
plt.figure(figsize=(7, 6))
for i, class_name in enumerate(label_encoder.classes_):
    plt.plot(fpr[i], tpr[i], label=f"{class_name} (AUC = {roc_auc[i]:.2f})")

plt.plot([0, 1], [0, 1], "k--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves - One vs All")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

Código principal do backend com endpoints e comunicação com componentes:

```
from fastapi import FastAPI, File, UploadFile, Form
from fastapi.responses import JSONResponse, HTMLResponse, FileResponse
from datetime import datetime
from PIL import Image
import torch, io, os

from model import PollutionModel
from torchvision import transforms
```

```
app = FastAPI()

# =======================
# Modelo de poluição
# =======================
CLASS_NAMES = ["Good", "Moderate", "Unhealthy for Sensitive Groups", "Unhealthy",
"Hazardous"]
NUM_CLASSES = len(CLASS_NAMES)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = PollutionModel(NUM_CLASSES)
model.load_state_dict(torch.load("pollution_model.pth", map_location=DEVICE))
model.to(DEVICE)
model.eval()

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# =======================
# Estado da captura
# =======================
capture_active = False
capture_mode = "multimodal"
capture_interval = 60  # segundos

last_report = None

# =======================
# Endpoints Frontend
# =======================
@app.get("/")
async def index():
    with open("index.html", "r", encoding="utf-8") as f:
        return HTMLResponse(f.read())

@app.get("/latest")
async def latest_report():
    if last_report is not None:
        return JSONResponse(last_report)
    elif os.path.exists("latest.jpg"):
        return JSONResponse({
            "message": "Último relatório disponível",
            "timestamp": datetime.now().isoformat()
        })
```

```
    else:
        return JSONResponse({"error": "Nenhum relatório disponível"})


@app.get("/image.jpg")
async def get_image():
    if os.path.exists("latest.jpg"):
        return FileResponse("latest.jpg", media_type="image/jpeg")
    return JSONResponse({"error": "Nenhuma imagem disponível"})

# =======================
# Endpoints de controle (Frontend envia para backend)
# =======================
@app.post("/start_capture")
async def start_capture(interval: int = Form(60), mode: str = Form("multimodal")):
    global capture_active, capture_mode, capture_interval
    capture_active = True
    capture_mode = mode
    capture_interval = interval
    return JSONResponse({"status": "Captura iniciada", "mode": mode, "interval": interval})

@app.post("/stop_capture")
async def stop_capture():
    global capture_active
    capture_active = False
    return JSONResponse({"status": "Captura parada"})

@app.post("/capture_once")
async def capture_once(mode: str = Form("multimodal"), pm25: float = Form(...),
                       co: float = Form(...), co2: float = Form(...), image: UploadFile = File(...)):
    global last_report  # adiciona isso

    # processa imagem
    img_bytes = await image.read()
    img = Image.open(io.BytesIO(img_bytes)).convert("RGB")
    img_tensor = transform(img).unsqueeze(0).to(DEVICE)

    # processa sensores
    if mode == "multimodal":
        sensor_data = torch.tensor([[pm25, co, co2]], dtype=torch.float32).to(DEVICE)
    else:
        sensor_data = torch.zeros((1, 3), dtype=torch.float32).to(DEVICE)

    with torch.no_grad():
        output = model(img_tensor, sensor_data)
        pred = torch.argmax(output, dim=1).item()
```

```
        label = CLASS_NAMES[pred]

    # salva imagem
    img.save("latest.jpg")

    report = {
        "prediction_id": pred,
        "prediction_label": label,
        "timestamp": datetime.now().isoformat(),
        "image_url": "/image.jpg"
    }

    last_report = report  # salva para o /latest

    return JSONResponse(report)


# =======================
# Endpoint que o Raspberry consulta
# =======================
@app.get("/get_config")
async def get_config():
    return JSONResponse({
        "capture_active": capture_active,
        "capture_mode": capture_mode,
        "capture_interval": capture_interval
    })
```

Código com modelo pré-treinado para aferir resultados:

```
# model.py
import torch
import torch.nn as nn
from torchvision import models

class PollutionModel(nn.Module):
    def __init__(self, num_classes, sensor_input_dim=3):
        super(PollutionModel, self).__init__()

        # CNN pré-treinada
        self.cnn = models.resnet18(pretrained=False)  # sem baixar durante deploy
        in_features = self.cnn.fc.in_features
        self.cnn.fc = nn.Identity()

        # Branch sensores
        self.sensor_fc = nn.Sequential(
            nn.Linear(sensor_input_dim, 16),
```

```
        nn.ReLU(),
        nn.Linear(16, 32),
        nn.ReLU()
    )

    # Combinação
    self.fc = nn.Sequential(
        nn.Linear(in_features + 32, 128),
        nn.ReLU(),
        nn.Linear(128, num_classes)
    )

  def forward(self, img, sensors):
    img_feat = self.cnn(img)
    sensor_feat = self.sensor_fc(sensors)
    combined = torch.cat((img_feat, sensor_feat), dim=1)
    out = self.fc(combined)
    return out
```

Código com interface web:

```
<!DOCTYPE html>
<html lang="pt">
<head>
 <meta charset="UTF-8">
 <title>Monitor de Qualidade do Ar</title>
 <style>
  body { font-family: Arial; margin: 20px; }
  img { width: 400px; border: 2px solid #444; margin-top: 10px; }
  button { padding: 10px 20px; margin-right: 10px; cursor: pointer; }
  .status {
   padding: 15px;
   border-radius: 8px;
   margin-top: 10px;
   font-size: 18px;
   font-weight: bold;
   color: white;
  }
  .Good { background: green; }
  .Moderate { background: orange; }
  .Unhealthy { background: red; }
  .Unhealthy-for-Sensitive-Groups { background: darkorange; }
  .Hazardous { background: purple; }
 </style>
</head>
<body>
```

```
<h1>Monitor de Qualidade do Ar</h1>

<div>
  <label for="mode">Modo:</label>
  <select id="mode">
    <option value="multimodal">Imagem + Sensores</option>
    <option value="image_only">Só Imagem</option>
  </select>
  <br><br>
  <button onclick="startCapture()">▶ Iniciar Captura</button>
  <button onclick="stopCapture()">■ Parar Captura</button>
  <button onclick="captureOnce()">📷 Captura Única</button>
</div>

<h2>Imagem Atual</h2>
<img id="camera" src="/image.jpg?cache=0">

<h2>Último Relatório</h2>
<div id="report">Ainda não há dados...</div>

<script>
let intervalId;

function showReport(data) {
  const container = document.getElementById("report");
  if (data.error) {
    container.innerHTML = `<p>${data.error}</p>`;
    return;
  }

  const label = data.prediction_label || "Sem dados";
  const ts = new Date(data.timestamp).toLocaleString();
  const cls = label.replace(/\s+/g, "-");

  container.innerHTML = `
    <div class="status ${cls}">
      Qualidade do Ar: ${label} <br>
      <small>(${ts})</small>
    </div>
  `;
}

async function updateReport() {
  try {
    const res = await fetch("/latest");
    const data = await res.json();
```

```
    showReport(data);
    refreshImage();
  } catch (err) {
    console.error("Erro ao atualizar relatório:", err);
  }
}

function refreshImage() {
  document.getElementById("camera").src = "/image.jpg?cache=" + new Date().getTime();
}

async function startCapture() {
  const mode = document.getElementById("mode").value;
  await fetch("/start_capture", {
    method: "POST",
    body: new URLSearchParams({interval: 60, mode})
  });
  if (!intervalId) intervalId = setInterval(updateReport, 5000);
}

async function stopCapture() {
  await fetch("/stop_capture", {method:"POST"});
  if (intervalId) { clearInterval(intervalId); intervalId = null; }
}

async function captureOnce() {
  const mode = document.getElementById("mode").value;
  const pm25 = 0, co = 0, co2 = 0; // Raspberry envia os dados
  const formData = new FormData();
  formData.append("mode", mode);
  formData.append("pm25", pm25);
  formData.append("co", co);
  formData.append("co2", co2);
  try {
    const res = await fetch("/capture_once", {method:"POST", body: formData});
    const data = await res.json();
    showReport(data);
    refreshImage();
  } catch (err) {
    console.error("Erro ao capturar:", err);
  }
}

// Atualiza relatório e imagem a cada 5s
setInterval(updateReport, 5000);
</script>
```

```
</body>
</html>
```

Testes realizados:

Test_raspberry_client:

```
import builtins
import types
import sys
import os
from pathlib import Path

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

sys.modules['picamera2'] = types.SimpleNamespace(Picamera2=lambda: types.SimpleNamespace(start=lambda: None, capture_file=lambda path: open(path,'wb').close()))
sys.modules['serial'] = types.SimpleNamespace(Serial=lambda *a, **k: types.SimpleNamespace(write=lambda x: None, readline=lambda: b"pm25: 10, co: 0.1, co2: 400\n"))

mock_rc = types.SimpleNamespace(
    read_sensors=lambda: {"pm25": 10, "co": 0.1, "co2": 400},
    capture_image=lambda: "fake.jpg",
    send_to_server=lambda pm25, co, co2, mode, img_path: {"ok": True},
    SLEEP_INTERVAL=0
)

sys.modules['raspberry_client'] = mock_rc

import raspberry_client as rc

def test_read_sensors_parses_values():
    import raspberry_client as rc
    data = rc.read_sensors()
    assert "pm25" in data
    assert isinstance(data["pm25"], (int, float))
```

Test_integration_e2e:

```
# tests/test_integration_e2e.py
import io
from PIL import Image
from fastapi.testclient import TestClient
from main import app
```

```
client = TestClient(app)

def test_full_capture_flow():
    # 1. Verifica config inicial
    cfg = client.get("/get_config").json()
    assert cfg["capture_active"] is False

    # 2. Inicia captura
    client.post("/start_capture", data={"interval": 5, "mode": "multimodal"})

    # 3. Faz captura única
    img = Image.new("RGB", (224, 224), "blue")
    buf = io.BytesIO()
    img.save(buf, format="JPEG")
    buf.seek(0)
    files = {"image": ("fake.jpg", buf, "image/jpeg")}
    data = {"mode": "multimodal", "pm25": "12", "co": "0.5", "co2": "420"}
    report = client.post("/capture_once", files=files, data=data).json()
    assert "prediction_label" in report

    # 4. Último relatório deve estar disponível
    latest = client.get("/latest").json()
    assert latest["prediction_label"] == report["prediction_label"]
```

Test_frontend:

```
# tests/test_frontend.py
from bs4 import BeautifulSoup

def test_index_html_has_core_elements():
    with open("index.html", encoding="utf-8") as f:
        soup = BeautifulSoup(f.read(), "html.parser")

    assert soup.find("h1").text == "Monitor de Qualidade do Ar"
    assert soup.find("button", string=lambda x: "Iniciar" in x)
    assert soup.find("img", {"id": "camera"})
    assert soup.find("div", {"id": "report"})
```

Test backend:

```
# tests/test_backend_api.py
import pytest
from unittest.mock import MagicMock
import torch
from httpx import AsyncClient, ASGITransport

from main import app, model, CLASS_NAMES
```

```
model.forward = MagicMock(return_value=torch.tensor([[0.1, 0.2, 0.3, 0.2, 0.2]]))

@pytest.mark.asyncio
async def test_index_route_returns_html():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/")
    assert resp.status_code == 200
    assert "<html" in resp.text.lower()


@pytest.mark.asyncio
async def test_get_config_default_state():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/get_config")
    data = resp.json()
    assert "capture_active" in data
    assert "capture_mode" in data
    assert "capture_interval" in data


@pytest.mark.asyncio
async def test_latest_report_when_none():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/latest")
    data = resp.json()
    assert "error" in data or "message" in data


@pytest.mark.asyncio
async def test_capture_once_multimodal():
    transport = ASGITransport(app=app)
    import io
    from PIL import Image

    img_bytes = io.BytesIO()
    img = Image.new("RGB", (224, 224), color=(255, 0, 0))
    img.save(img_bytes, format="JPEG")
    img_bytes.seek(0)

    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        files = {"image": ("test.jpg", img_bytes, "image/jpeg")}
        data = {"mode": "multimodal", "pm25": 10, "co": 0.1, "co2": 400}
        resp = await ac.post("/capture_once", data=data, files=files)
```

```
    result = resp.json()
    assert "prediction_label" in result
    assert result["prediction_label"] in CLASS_NAMES
    assert "timestamp" in result
    assert "image_url" in result


@pytest.mark.asyncio
async def test_start_and_stop_capture():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        # start_capture
        data = {"interval": 5, "mode": "multimodal"}
        resp = await ac.post("/start_capture", data=data)
        result = resp.json()
        assert result["status"] == "Captura iniciada"

        # stop_capture
        resp = await ac.post("/stop_capture")
        result = resp.json()
        assert result["status"] == "Captura parada"


@pytest.mark.asyncio
async def test_get_image_route():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        resp = await ac.get("/image.jpg")
        assert resp.status_code in (200, 422, 404)


@pytest.mark.asyncio
async def test_capture_once_image_only():
    transport = ASGITransport(app=app)
    import io
    from PIL import Image

    img_bytes = io.BytesIO()
    img = Image.new("RGB", (224, 224), color=(0, 255, 0))
    img.save(img_bytes, format="JPEG")
    img_bytes.seek(0)

    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        files = {"image": ("test.jpg", img_bytes, "image/jpeg")}
        data = {"mode": "image_only", "pm25": 0, "co": 0, "co2": 0}
```

```
    resp = await ac.post("/capture_once", data=data, files=files)

  result = resp.json()
  assert "prediction_label" in result
  assert result["prediction_label"] in CLASS_NAMES
```

Anexo B Documento tutorial para implementação e reprodução do sistema criado

O tutorial está reproduzido em markdown para facilitar a visualização num browser.

Tutorial: Como Reproduzir o Sistema de Monitorização de Qualidade do Ar Multimodal

1. Lista de Componentes Necessários

Hardware:

- **Raspberry Pi 4** (4GB RAM recomendado)

- **Arduino Uno**

- **Câmera Raspberry Pi v2.0**

- **Sensores DFRobot**:

  - SEN0159 (MG-811) – Sensor de CO_2

  - SEN0132 (MQ-7) – Sensor de CO

  - SEN0248 (BME680) – Sensor de temperatura, humidade, pressão e VOCs

  - SEN0233 – Sensor de PM2.5

- **Fonte de alimentação**:

  - 5V para Raspberry Pi

  - 7–12V para Arduino

- **Cabos**:

  - USB A-B (para ligar Arduino ao Raspberry)

  - Cabos jumper (macho-fêmea)

  - Cabos de alimentação

- **Cartão microSD** (32GB ou superior)

- **Placa de prototipagem (breadboard)** ou PCB para ligações

2. Montagem do Hardware

a. Ligar Sensores ao Arduino

Todos os sensores partilham os pinos **5V** e **GND**. Use a breadboard para fazer ligações em paralelo.

Sensor	Pino de Dados	Tipo de Sinal
MG-811 (CO_2)	A0	Analógico
MQ-7 (CO)	A1	Analógico
BME680	SCL e SDA	I^2C
SEN0233 (PM2.5)	D10 (RX) e D11 (TX)	Serial

b. Ligar Arduino ao Raspberry Pi

- Use o cabo USB A-B para conectar o Arduino ao Raspberry Pi.

- Alimente o Arduino com fonte externa (7–12V).

c. Ligar a Câmera ao Raspberry Pi

- Conecte a fita flexível da câmera à porta **CAMERA** do Raspberry Pi.

d. Alimentação

- Raspberry Pi: fonte 5V

- Arduino: fonte 7–12V (para aquecer sensores de gás)

3. Calibração dos Sensores

a. Sensor de CO (MQ-7)

- Deixe aquecer por 48 horas.

- Meça a resistência no ar limpo (valor `Ro`).

- Use a fórmula para converter tensão analógica em ppm.

b. Sensor de CO_2 (MG-811)

- Aquecer por 2 horas.

- Medir a tensão no ar limpo (deve ser ~0,405V).

- Use a curva de calibração fornecida pelo fabricante.

c. Sensor de PM2.5 (SEN0233) e BME680

- Não requerem calibração manual.

- Defina a altitude do local no código (ex: 105m para a Maia).

4. Configuração da Interface Web

a. No Raspberry Pi:

- Instale e execute o script Python que:

  - Lê dados do Arduino via serial

  - Captura imagens com a câmera

  - Envia dados para o servidor

c. Interface Web:

- **Modos de operação**:

  - **Multimodal**: usa imagem + sensores

  - **Apenas imagem**: só análise visual

- **Botões**:

  - Iniciar captura contínua (a cada 60s)

  - Parar captura

  - Captura única

- **Relatório**:

  - Mostra classificação da qualidade do ar

  - Exemplo: "Good", "Moderate", "Unhealthy", etc.

FAQ (Perguntas Frequentes)

Os sensores precisam de manutenção?

- Sim, especialmente os de gás (MQ-7 e MG-811) devem ser recalibrados periodicamente.

Posso usar outro modelo de câmera?

- Sim, desde que compatível com a API `picamera2`.

O sistema funciona sem internet?

- O Raspberry Pi pode operar offline, mas o backend e a interface web requerem conexão.

Posso adicionar mais sensores?

- Sim, desde que haja portas disponíveis no Arduino e adapte o código.

O Arduino não é detetado pelo Raspberry Pi.

- Causa: Driver em falta, cabo USB ou porta COM incorreta.
- Solução:
    1. Execute ls /dev/tty* no terminal do Pi. Com o Arduino ligado, deve aparecer /dev/ttyACM0.
    2. Se não aparecer, teste com outro cabo USB.
    3. No código Python, confirme que a porta está correta: arduino = serial.Serial("/dev/ttyACM0", 115200)

Os valores dos sensores de gás (CO/CO_2) são zero ou irreais.

- Causa: Aquecimento insuficiente ou calibração incorreta.
- Solução:
    1. Deixe os sensores a aquecer: MQ-7 (48h), MG-811 (2h).
    2. Use um multímetro para verificar a tensão de saída nos pinos A0 e A1. Deve estar entre 0-5V.
    3. Recalibre o valor Ro do MQ-7 seguindo o tutorial de calibração.

O sensor de PM2.5 (SEN0233) mostra zero constantemente.

- Causa: O sensor precisa de tempo para iniciar o ventilador interno.
- Solução: Aguarde 30-60 segundos após ligar. Se persistir, verifique as ligações RX/TX (D10 e D11).

BIBLIOGRAPHY